Project Report

On

SATYA-SCORE: AN AI HALLUCINATION DETECTION & SCORING FRAMEWORK

Submitted to

Prof. Ram Meghe Institute of Technology and Research,

(An Autonomous Institute)

Affiliated to Sant Gadge Baba Amravati University, Amravati

for partial fulfillment of the requirement for the degree of

BACHELOR OF TECHNOLOGY

in

Artificial Intelligence and Data Science

Submitted By

Vivek Yeshwantrao Borkar

Piyush Devidas Nande

Aasawari Sahebrao Jumade

Arunima Tridip Wankhade

Under the Guidance of

Dr. R. R. Karwa

Department of Artificial Intelligence and Data Science

Prof. Ram Meghe Institute of Technology & Research, Badnera - Amravati

2026-2027

Department of Artificial Intelligence and Data Science
Prof. Ram Meghe Institute of Technology & Research, Badnera - Amravati
(An Autonomous Institute)
2026 - 2027

CERTIFICATE

*This is to certify that the **Project (7SA08)** entitled*

Satya-Score: An AI Hallucination Detection & Scoring Framework

is a bonafide work and it is submitted to the Prof. Ram Meghe Institute of Technology and Research (An Autonomous Institute), affiliated to
Sant Gadge Baba Amravati University, Amravati

By

Vivek Yeshwantrao Borkar
Piyush Devidas Nande
Aasawari Sahebrao Jumade
Arunima Tridip Wankhade

in the partial fulfillment of the requirement for the degree of ***Bachelor of Technology*** *in* ***Artificial Intelligence and Data Science,***
during the academic year ***2026-2027*** *under my guidance.*

Dr. R. R. Karwa
Guide
Department of AI & DS
Prof. Ram Meghe Institute of Technology & Research, Badnera - Amravati

Dr. S. R. Gupta
Head,
Department of AI & DS
Prof. Ram Meghe Institute of Technology & Research, Badnera - Amravati

External Examiner

_____

ACKNOWLEDGEMENT

With immense pleasure and deep gratitude, we would like to express our sincere thanks to all those who have supported and guided us throughout the completion of this project. This work stands as a testament to the inspiration, motivation, and technical assistance provided by many individuals. Without their continuous support and encouragement, this project would not have been possible.

We are profoundly grateful to our respected guide, **Dr. R. R. Karwa**, for valuable guidance, constant supervision, constructive suggestions, and for providing the necessary resources required for the successful completion of this project. The patience, encouragement, and insightful feedback we received greatly contributed to the quality of our work.

We would also like to extend our sincere appreciation to **Dr. S. R. Gupta**, Head of the Department of Artificial Intelligence and Data Science, for providing us with the opportunity and encouragement to undertake this project. We are equally thankful to all the faculty members of the Department of Artificial Intelligence and Data Science for their kind cooperation, continuous encouragement, and academic support.

We owe a special note of gratitude to the staff members of the Department of Artificial Intelligence and Data Science for their direct and indirect assistance throughout this project.

We also express our heartfelt thanks to our colleagues for their collaboration, teamwork, and valuable contributions in developing this project. Their support and cooperation played a significant role in its successful completion. We are also thankful to the authors whose research work we referred to and used in this project, and to the people who maintain the open resources it relies on: Wikipedia, Wikidata, World Bank Open Data, the WHO Global Health Observatory and the published fact-checks.

Finally, we extend our deepest gratitude to our parents, friends, and well-wishers for their constant encouragement, moral support, and understanding. Above all, we thank the Almighty God for guiding us and blessing us with the strength and wisdom to accomplish this work.

1. Vivek Yeshwantrao Borkar
2. Piyush Devidas Nande
3. Aasawari Sahebrao Jumade
4. Arunima Tridip Wankhade

ABSTRACT

Chatbots based on large language models (LLMs) write fluent answers that sometimes contain false statements, and readers cannot easily tell which sentence is wrong; for Hindi, checking tools are scarce. Satya-Score is a framework that detects and scores such hallucinations in English and Hindi text one claim at a time. A claim that official statistics (World Bank, WHO) or the Wikidata knowledge graph can settle is decided by that source, with the correct value shown; otherwise a matching published fact-check is used. For the remaining claims, Version 2 adds an LLM cross-check: a multilingual inference model reads retrieved Wikipedia passages, an open-weight LLM (gpt-oss-20b, run on the project laptop) judges the claim, and the LLM's verdict counts only when the passage evidence agrees. Each claim receives a verdict (Verified, Uncertain or Hallucinated), a score, its evidence and the wrong words marked, in a secure web application.

Every change was tested against criteria fixed in advance. Structured sources caught all 137 altered country statistics without contradicting a true figure, and a pre-registered ablation showed the knowledge graph raising AUC from 0.704 to 0.784. On a held-out sample of 620 English and Hindi items from FEVER, Mu-SHROOM, TruthfulQA, X-FACT and Hindi Wikipedia, evaluated once, the cross-check caught 47% of false items against 25% without it, passed only 6% of them as Verified against 22%, and raised false alarms on 13% of true items against 11%. The LLM alone caught 86% but flagged 28% of true items. The cross-check met three of the four acceptance criteria, narrowly missing a doubling of catches, and its gains were largest in Hindi.

Table of Contents

Chapter No		Content / Title	Page No
		List of Tables	*vii*
		List of Figures	*viii*
		List of Screenshots	*ix*
		Notations	*x*
		Abbreviations	*xi*
1.		**Introduction**	**1-4**
	1.1	Overview	1
	1.2	Motivation	1
	1.3	Problem Definition	1
	1.4	Project Goal and Objectives	2
	1.5	Contribution	3
	1.6	Organization of Report	3
2.		**Literature Review**	**5-8**
	2.1	Fact Verification and Claim Checking	5
	2.2	Hallucination Detection in LLM Output	5
	2.3	Grounding Checkers	6
	2.4	Multilingual and Hindi Fact-Checking	6
	2.5	Knowledge Graphs and Cross-Lingual Retrieval	6
	2.6	Summary of the Literature	7
	2.7	Research Gap	7
3.		**System Architecture / Analysis**	**9-14**
	3.1	Existing System and its Limitations	9
	3.2	Proposed System	9
	3.3	Requirement Analysis	10
	3.4	Hardware and Software Requirements	10
	3.5	Feasibility Study	11
	3.6	Overall System Architecture	12
4.		**System Design**	**15-26**
	4.1	Block Diagram	15
	4.2	Data Flow Diagrams	16
	4.3	Module Design	17
	4.4	Algorithms	18
	4.5	Database Design	24
	4.6	Flowchart of a Fact-Check Request	24
5.		**Implementation and Results**	**27-44**

Chapter No		Content / Title	Page No
	5.1	Technologies Used	27
	5.2	Implementation of the Modules	27
	5.3	Web Application	28
	5.4	Datasets and Annotation	31
	5.5	Evaluation Method	33
	5.6	Results on Real Chatbot Claims and Unseen Data	34
	5.7	Results on Public Benchmarks	34
	5.8	Results of the LLM Cross-Check (Version 2)	36
	5.9	Results of the Structured Sources	39
	5.10	Changes Tested and Not Adopted	40
	5.11	Performance	41
	5.12	Discussion and Limitations	41
6.		**Conclusions and Future Scope**	**45**
	6.1	Conclusion	45
	6.2	Future Scope	45
		References	**46**
		List of Publications	**50**
		Resume	**51**

List of Tables

Table No.	Name of Table	Page No.
2.1	Comparison of existing systems	7
3.1	Hardware requirements	11
3.2	Software requirements	11
4.1	Main modules	17
4.2	Database tables	24
5.1	Technologies used	27
5.2	Datasets	32
5.3	Results of the running system on chatbot claims and unseen data	34
5.4	FEVER sample: live system and inference only	35
5.5	Mu-SHROOM test sets: span IoU and correlation	35
5.6	Held-out comparison of Version 1 and Version 2 (620 items)	36
5.7	Results per set, Version 1 → Version 2 (percent of items)	37
5.8	Ablation on the held-out sample (exploratory)	37
5.9	Development rounds of the cross-check on held-out samples (Version 2 against Version 1)	38
5.10	Mu-SHROOM span IoU on the held-out answers	38
5.11	Results of the structured sources	39
5.12	Changes tested and the decisions taken	40
5.13	Performance on the laptop (RTX 4050 GPU)	41

List of Figures

Figure No.	Name of Figure	Page No.
3.1	Overall system architecture	13
4.1	Block diagram of the pipeline	15
4.2	Data flow diagram, level 0	16
4.3	Data flow diagram, level 1	17
Listing 4.1	Evidence rule (member3_backend/evidence_rules.py)	19
Listing 4.2	Two-agency agreement for statistics (member2_data/official_stats.py)	20
Listing 4.3	Direction-aware fact-check matching (member3_backend/factcheck_rules.py)	20
Listing 4.4	Claim scoring (member1_model/cmhsa.py)	21
4.4	Decision of the LLM cross-check for one claim	23
Listing 4.5	Agreement rule of the cross-check (member3_backend/crosscheck.py)	23
4.5	Flowchart of a fact-check request	25

List of Screenshots

Screenshot No.	Name of Screenshot	Page No.
5.1	Input page with an example answer	28
5.2	Results page: whole-text verdict and highlighted claims	29
5.3	Claim cards with the correct fact from Wikidata	29
5.4	A Hindi claim decided by a published fact-check	29
5.5	History of saved checks (account e-mail hidden)	30
5.6	Version 2: wrong words marked and the verdict of each reader	31

Notations

Symbol	Meaning
s	Passage (evidence) score of a claim from the inference model, between 0 and 1
e(p), c(p)	Probability that passage p entails / contradicts the claim
j	Verdict of the LLM's first reading (Verified, Uncertain or Hallucinated)
α, β, γ	CMHSA weights of the similarity, inference and knowledge-graph signals
HalluScore	CMHSA score of a claim (0 = contradicted, 1 = supported)
SemSim, NLI, KG	Similarity, inference and knowledge-graph signals of a claim
κ	Cohen's kappa, agreement between two annotators
p	p-value of a statistical test (exact McNemar test)
n×	Ratio of catches between two versions (e.g. 1.85×)

Abbreviations

Abbreviation	Full form
AI	Artificial Intelligence
API	Application Programming Interface
AUC	Area Under the ROC Curve
CMHSA	Composite Multilingual Hallucination Scoring Algorithm
CSV	Comma-Separated Values
DFD	Data Flow Diagram
FAISS	Facebook AI Similarity Search
FEVER	Fact Extraction and VERification (dataset)
GPU	Graphics Processing Unit
IoU	Intersection over Union
IPR	Intellectual Property Rights
JSON	JavaScript Object Notation
KG	Knowledge Graph
LLM	Large Language Model
mDeBERTa	Multilingual Decoding-enhanced BERT with disentangled attention
MNLI / XNLI	Multi-Genre / Cross-lingual Natural Language Inference corpora
Mu-SHROOM	Multilingual Shared-task on Hallucinations and Related Observable Overgeneration Mistakes
NLI	Natural Language Inference
NLP	Natural Language Processing
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
ROC	Receiver Operating Characteristic
SemEval	International Workshop on Semantic Evaluation
SQL	Structured Query Language
UI	User Interface
WHO	World Health Organization
XLM-R	Cross-lingual Language Model (RoBERTa)

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

Chatbots built on large language models (LLMs), for example GPT-4 [1] and Llama [2], now answer everyday questions on health, public services, science and history. They produce text by repeatedly choosing a likely next word; they do not consult a record of facts while writing. As a result, an answer can be grammatical, confident and still contain statements that are invented or wrong. Such output is usually called hallucination [3], [4]. Studies of long model-written answers have found that a noticeable share of their individual facts is not supported by any reliable source [5].

Satya-Score ("satya" means truth) is a framework for finding these errors. It breaks an answer into individual factual claims, checks each claim against the most authoritative source available, and reports for every claim a verdict, a score and the evidence behind it. Version 2 of the system adds an open-weight LLM as a second reader whose verdict counts only when the retrieved evidence agrees with it.

1.2 MOTIVATION

A wrong date, name or figure inside a long and otherwise correct answer is easy to miss, and checking every sentence by hand removes the benefit of using an assistant. Automatic checkers exist, but most work only for English, give a single score for a whole passage, or depend on another large model whose own judgement may be wrong. Hindi is spoken by hundreds of millions of people and appears more and more in chatbot output, yet a Hindi reader has almost no tool that points to the false statement in an answer and explains why it is false. This gap motivated the project.

1.3 PROBLEM DEFINITION

Given an answer produced by an LLM in English or Hindi, identify its factual claims; decide for each claim whether reliable sources support it, contradict it or cannot settle it; show the evidence and, where a structured source holds it, the correct

fact; and do this quickly enough to be useful while reading, without marking true statements as false.

1.4 PROJECT GOAL AND OBJECTIVES

The goal of the project is a bilingual, explainable checker that tells a reader which statements of an AI answer are true, false or unsettled, and why. The objectives are:

i) To analyse hallucination in LLM answers.

ii) To compare existing hallucination-detection methods.

iii) To design a claim-level detection framework for English and Hindi.

iv) To integrate an LLM with retrieved evidence so that neither decides alone.

v) To develop a secure web application that explains each verdict.

vi) To evaluate the system on public benchmarks with pre-set criteria.

1.4.1 Scope of the project

- **Languages:** English and Hindi text, typed, pasted or uploaded as a .txt or .pdf file.
- **Claims:** statements of fact. Questions, greetings, advice and opinions are recognised and not checked.
- **Sources:** English and Hindi Wikipedia, Wikidata, World Bank Open Data, the WHO Global Health Observatory and published fact-checks found through the Google Fact Check Tools API; in Version 2, also an open-weight LLM (gpt-oss-20b) whose verdict counts only with the evidence.
- **Output:** a verdict, score and explanation for every claim with the wrong words marked, an overall verdict for the text, and a history of checks for signed-in users.
- **Outside the scope:** other languages, images and audio, correcting the chatbot itself, and commercial use (the WHO data licence allows non-commercial use only).

1.5 CONTRIBUTION

To our knowledge, Satya-Score is the first claim-level checker that combines official statistics, a knowledge graph, published fact-checks and an LLM for English and Hindi, in which the LLM can never decide a claim alone. The project contributes:

1. **An agreement-gated LLM cross-check:** a claim is Verified only when the passage evidence and the LLM agree, and Hallucinated only when the LLM names a wrong fact that a second, passages-only reading does not find stated. Used alone, the same LLM raised false alarms on 28% of true items; under this rule, 13% (Section 5.8).
2. **Source routing with the correct value:** each claim is answered by the most trustworthy source that covers it (statistics, knowledge graph, fact-check, text), and a structured source also gives the correct value.
3. **Marking of the wrong words** in the text, instead of flagging whole sentences.
4. **An evaluation with criteria fixed in advance** on public English and Hindi benchmarks and Hindi sets built for this study, in which every held-out sample was used once and failures are reported.
5. **An open, local system:** all models are open and run on a student laptop with a 6 GB GPU, together with a secure web application.

1.6 ORGANIZATION OF REPORT

Chapter 2 reviews related work and identifies the research gap. Chapter 3 analyses the existing approaches and the requirements and presents the overall architecture. Chapter 4 gives the design: block diagram, data flow diagrams, modules, algorithms, database and flowchart. Chapter 5 describes the implementation and reports the results of both versions, including the changes that were tested and rejected and the limitations. Chapter 6 concludes and outlines the future scope of the work.

CHAPTER 2

LITERATURE REVIEW

2.1 FACT VERIFICATION AND CLAIM CHECKING

Automatic fact verification became a standard task with FEVER [6], in which a system receives a claim, finds evidence in Wikipedia and labels the claim as supported, refuted or lacking enough information. Later work asked whether a generated text is consistent with a given source. The TRUE study [7] compared many such consistency metrics on a common footing, and AlignScore [8] trained a single alignment function over many tasks for the same purpose. These methods assume that the right evidence is already available; finding it is a separate problem.

2.2 HALLUCINATION DETECTION IN LLM OUTPUT

Benchmarks such as TruthfulQA [9] and HaluEval [10] measure how often models produce false or unsupported statements. FActScore [5] splits a generated biography into atomic facts and checks each against Wikipedia, and SAFE [11] uses an LLM agent that searches the web for every fact. Both show that checking individual facts is more informative than scoring a whole answer, but both are built for English and rely on large models for splitting or judging. SelfCheckGPT [12] needs no external source: it samples several answers and treats disagreement between them as a warning, which cannot catch an error the model repeats consistently. Other work operates at a finer grain: RefChecker [13] checks claim triplets against a reference, ANAH [14] supplies sentence-level analytical annotations for training hallucination annotators, FAVA [15] detects and edits erroneous spans, and MIND [16] uses a model's internal states. A known weakness of inference models is that they can rely on word overlap rather than meaning [17]; this weakness reappears in the error analysis of Chapter 5. Strong LLMs are also widely used as judges, and they agree with human preferences in many tasks [18]; a judge, however, shares the blind spots of its own model family, and when used to check facts it can treat missing evidence as proof of falsehood. Version 2 of Satya-Score therefore lets an LLM count only together with retrieved evidence.

2.3 GROUNDING CHECKERS

A grounding checker decides whether a sentence is supported by a supplied document. MiniCheck [19] reaches accuracy close to GPT-4 on this task at a much smaller size and cost, and AlignScore [8] was the strongest small checker before it. Retrieval-augmented generation [20] takes a related route, giving the model retrieved documents while it writes; it can reduce errors in a system one controls, but it cannot audit text that a third-party chatbot has already produced.

2.4 MULTILINGUAL AND HINDI FACT-CHECKING

X-FACT [21] collected short statements in 25 languages, Hindi among them, labelled by professional fact-checkers, and showed that multilingual fact-checking remains hard. The SemEval-2025 Mu-SHROOM shared task [22] posed span-level hallucination detection in 14 languages, Hindi among them. Participating systems used, for example, an ensemble of LLMs that vote on candidate spans [23], natural-language inference with a model fine-tuned on synthetic data [24], and semantic-role decomposition with retrieved context [25]; none of these three uses a knowledge graph. SHROOM-CAP [26] extended the task to scientific publications in nine languages, and BHRAM-IL [27] provides a benchmark that measures hallucination in Hindi, Gujarati, Marathi and Odia as well as English. Multilingual encoders such as XLM-R [28] and MuRIL [29], the IndicNLPSuite resources [30], IndicSBERT [31] and IndicXNLI [32] supply building blocks for Indian languages, while inference models are commonly trained on MultiNLI [33] and its cross-lingual extension XNLI [34].

2.5 KNOWLEDGE GRAPHS AND CROSS-LINGUAL RETRIEVAL

A knowledge graph stores facts as explicit relations between entities. Wikidata [35] is open, multilingual and edited collaboratively. FactKG [36] verifies claims by reasoning over graph paths, and KGR [37] uses a graph to retrofit and correct a model's draft answer. MultiHal [38] is a multilingual benchmark that pairs questions with knowledge-graph paths in several European languages and reports that answers grounded in those paths score better on similarity and inference measures. Two recent surveys conclude that the benefit of knowledge graphs against hallucination is promising but not yet settled [39], [40]. For retrieval, sentence encoders such as

Sentence-BERT [41] and its multilingual distillation [42] map text into vectors that can be searched quickly with FAISS [43]; DeBERTaV3 [44] is a strong encoder family on which multilingual inference models are built.

2.6 SUMMARY OF THE LITERATURE

Table 2.1 summarises representative systems along the properties that matter for this project.

Table 2.1 Comparison of existing systems

System	Unit checked	Hindi	Evidence used	Gives correct fact
FActScore [5]	Atomic facts	No	Wikipedia passages	No
SAFE [11]	Atomic facts	No	Web search through an LLM agent	No
SelfCheckGPT [12]	Sentences	No	None (agreement among sampled answers)	No
MiniCheck [19]	Sentences	No	A supplied document	No
Mu-SHROOM systems [23], [24], [25]	Text spans	Yes	LLMs, inference models or retrieved context; no knowledge graph	No
MultiHal [38]	Benchmark (question answering)	No	Knowledge-graph paths	Not applicable
BHRAM-IL [27]	Benchmark	Yes	Not applicable	Not applicable
Satya-Score (this work)	Claims	Yes	Official statistics, Wikidata, published fact-checks, Wikipedia; an LLM that counts only with the evidence	Yes, when statistics or Wikidata decide; wrong words marked

2.7 RESEARCH GAP

Claim-level checking, Hindi coverage and knowledge-graph grounding have each been studied, but in separate lines of work. The multilingual detectors reviewed here do not use structured sources, and the graph-grounded work does not cover Hindi. None of the reviewed systems routes each claim to official statistics, a knowledge graph or a published fact-check before falling back on text, states the correct fact when it finds an error, and does this for Hindi. LLM judges catch more errors than inference models but raise many false alarms when used alone. Satya-Score addresses this

combination and, in Version 2, uses an LLM as one of two readers that must agree. Its components are existing public models and data sources; the contribution lies in how they are combined, in the safeguards placed on each source and on the LLM, and in an evaluation that reports failures as well as successes.

CHAPTER 3

SYSTEM ARCHITECTURE / ANALYSIS

3.1 EXISTING SYSTEM AND ITS LIMITATIONS

Today a reader who doubts a chatbot answer has three options: check each statement by hand, look for an article by a professional fact-checking organisation, or use one of the research tools reviewed in Chapter 2. Each has limits:

- Manual checking is slow and is rarely done for every sentence.
- Professional fact-checks cover popular rumours, not the particular facts in a particular answer.
- Most research tools handle English only, return one score for a whole text, or ask another LLM for the verdict, which can itself be wrong.
- Few tools explain a verdict with a source the reader can open, and fewer still state what the correct fact is.

3.2 PROPOSED SYSTEM

Satya-Score checks each claim separately and sends it to the strongest source able to settle it. Official statistics and the Wikidata knowledge graph are consulted first, because they hold exact values. If neither answers, a published fact-check of the same claim is used when one exists. Otherwise, passages from English and Hindi Wikipedia are retrieved and judged by a multilingual inference model under a conservative rule that prefers “Uncertain” to a false alarm. Version 2 adds a second reader for these claims: an open-weight LLM reads the claim and the passages, and a claim is Verified only when the passages and the LLM agree, and Hallucinated only when the LLM names a wrong fact that the retrieved passages, read again, do not state. Every verdict carries its evidence, the wrong words of a flagged claim are marked, and an error decided by a structured source is shown together with the correct value.

3.3 REQUIREMENT ANALYSIS

Functional requirements

FR1. Accept English or Hindi text, typed or pasted or uploaded as .txt or .pdf, with the language chosen by the user or detected from the script.

FR2. Extract self-contained factual claims and skip questions, greetings, advice and opinions.

FR3. Verify each claim against official statistics, Wikidata, published fact-checks and Wikipedia, and (Version 2) cross-check the remaining claims with an LLM.

FR4. Give each claim a score and a verdict (Verified, Uncertain or Hallucinated), with its source, its evidence, the wrong words marked and, when available, the correct fact.

FR5. Give the whole text an overall verdict and score.

FR6. Allow users to register, sign in, keep a history of checks and export results as CSV or JSON.

Non-functional requirements

- **Reliability:** true statements should rarely be marked Hallucinated; when the evidence is weak the system should answer Uncertain.
- **Performance:** a typical answer should be checked within seconds without the LLM, and within about half a minute with the LLM cross-check.
- **Security:** passwords and sessions must be protected and requests from other websites refused.
- **Transparency:** every verdict must name its source.
- **Portability:** the system must run on a laptop, with or without a GPU.

3.4 HARDWARE AND SOFTWARE REQUIREMENTS

Table 3.1 lists the hardware on which the system was developed and measured, and Table 3.2 the software. The memory figures were measured with the server running on the CPU.

Table 3.1 Hardware requirements

Component	Used for development	Needed to run the server
Processor	Intel Core i5-13450HX	A 64-bit processor (CPU-only operation works, more slowly)
Graphics	NVIDIA GeForce RTX 4050 Laptop GPU, 6 GB	Optional; speeds up the models
Memory	24 GB RAM	About 4 GB without the LLM (measured: 2.3 GB after loading, 2.9 GB at peak); 16 GB or more with the local LLM (13 GB model)
Storage	512 GB SSD	About 2 GB for models and index; 13 GB more for the local LLM
Network	Internet connection	Internet connection for the live look-ups and, optionally, the Groq and OpenRouter services

Table 3.2 Software requirements

Purpose	Software (version used)
Operating system	Windows 11 (development); Linux container image for cloud hosting
Back end	Python 3.13, FastAPI 0.115.6, Uvicorn 0.34.0, SQLite (Python standard library)
Models and search	PyTorch 2.13.0, Transformers 5.14.1, Sentence-Transformers 5.6.0, FAISS (faiss-cpu 1.14.3)
Language processing	spaCy 3.8.14 with en_core_web_sm, Indic NLP Library 0.92, pdfminer.six
LLM cross-check (Version 2)	gpt-oss-20b run with Ollama 0.40 on the laptop; optionally the same model through the Groq or OpenRouter API
Front end	React 18, TypeScript 5, Vite 5, Tailwind CSS 3, built with Node.js 24
Client	Any modern web browser

3.5 FEASIBILITY STUDY

- **Technical:** all models are open and run on a laptop GPU or CPU, including the 21-billion-parameter LLM, which Ollama [45] runs partly on the 6 GB GPU and partly on the CPU; the structured sources are reached through public web interfaces.
- **Economic:** the models, libraries and data sources are free of charge; the Google Fact Check Tools API needs only an API key, and the optional Groq

service has a free tier (8,000 tokens per minute, 1,000 requests per day), with a paid tier for faster use; OpenRouter, the second cloud service, charges a small amount per reading.

- **Operational:** users need only a web browser; results are shown in place, with links to the sources and the verdict of each reader.
- **Legal:** the models, including gpt-oss-20b [46], are released under the MIT or Apache-2.0 licences; Wikidata is CC0, World Bank data CC BY 4.0, Wikipedia text CC BY-SA 4.0, and WHO data CC BY-NC-SA 3.0 IGO, which limits the system to non-commercial use unless the WHO source is switched off.

3.6 OVERALL SYSTEM ARCHITECTURE

Fig. 3.1 shows the architecture of Version 2. The React web application sends the text to the FastAPI back end, which handles accounts and sessions and stores history in SQLite. The back end extracts the claims and consults the structured sources and fact-checks for each claim; a claim that none of them decides is scored by the inference model and cross-checked by the LLM, and the scored and explained result is returned. With the cross-check switched off (Version 1), the inference score decides directly. All figures in this report were drawn by the authors.

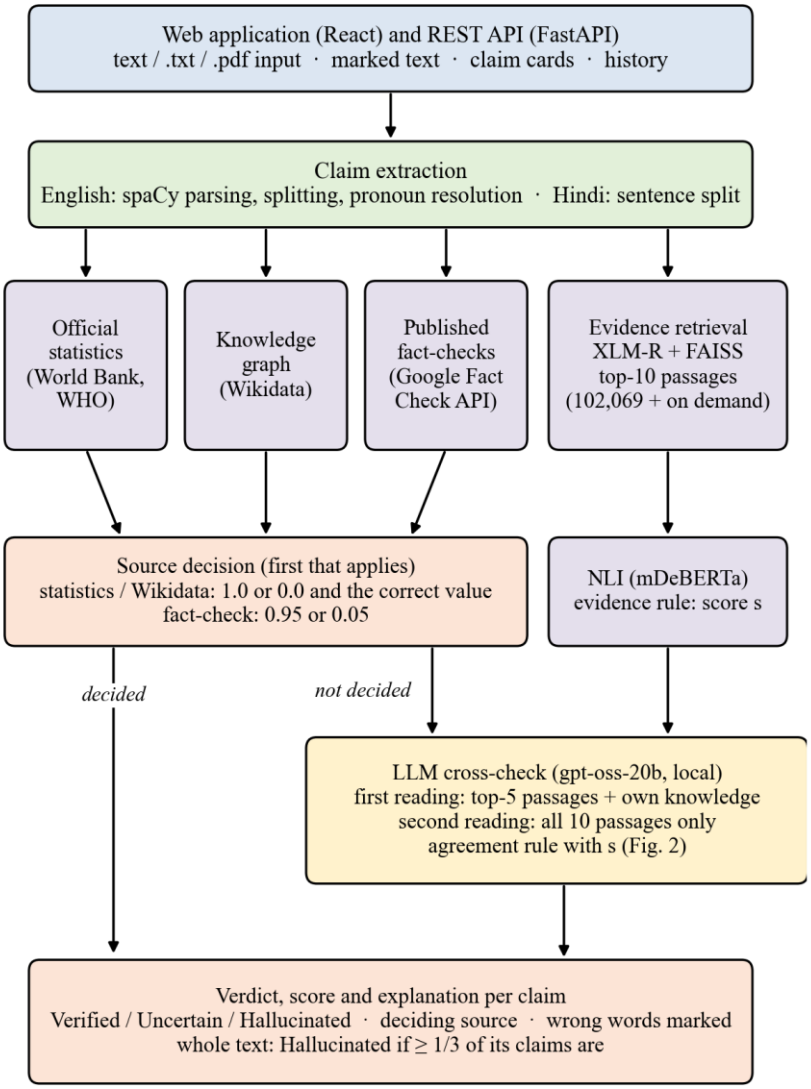


Fig. 3.1 Overall system architecture

CHAPTER 4

SYSTEM DESIGN

4.1 BLOCK DIAGRAM

Fig. 4.1 shows the processing blocks of a check and, in particular, the order in which the sources decide a claim. It shows Version 1; in Version 2, step (3) of the decision block, the evidence-rule score, is replaced by the LLM cross-check of Section 4.4.7 (Fig. 4.4).

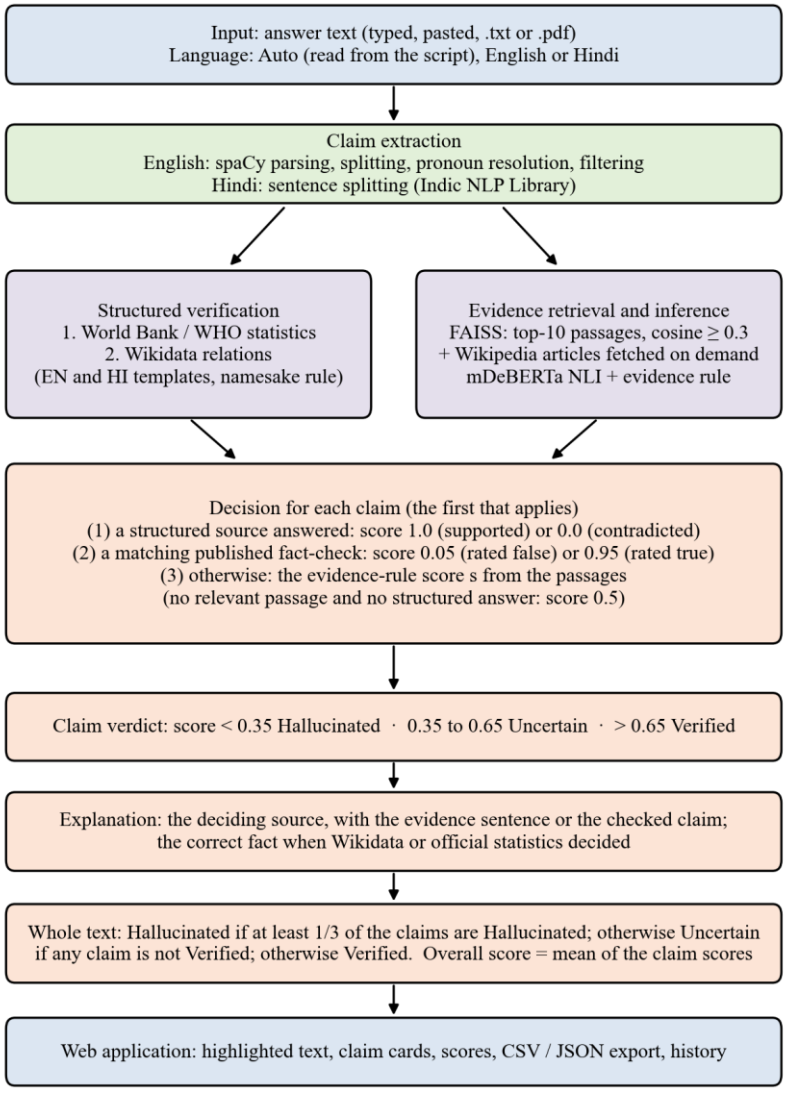


Fig. 4.1 Block diagram of the pipeline

4.2 DATA FLOW DIAGRAMS

Fig. 4.2 is the context-level (level 0) data flow diagram: answer text comes in from the user, evidence comes from four external sources, and checks are stored in the user database. Fig. 4.3 refines the system into six processes. Processes 2.0, 3.0 and 4.0 work on the claims produced by process 1.0; process 5.0 combines their results, and process 6.0 keeps accounts and history.

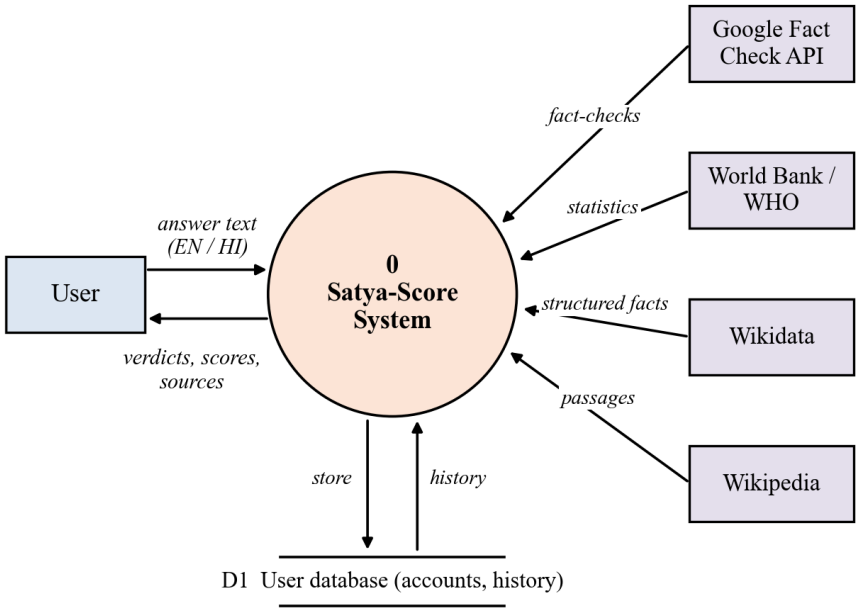


Fig. 4.2 Data flow diagram, level 0

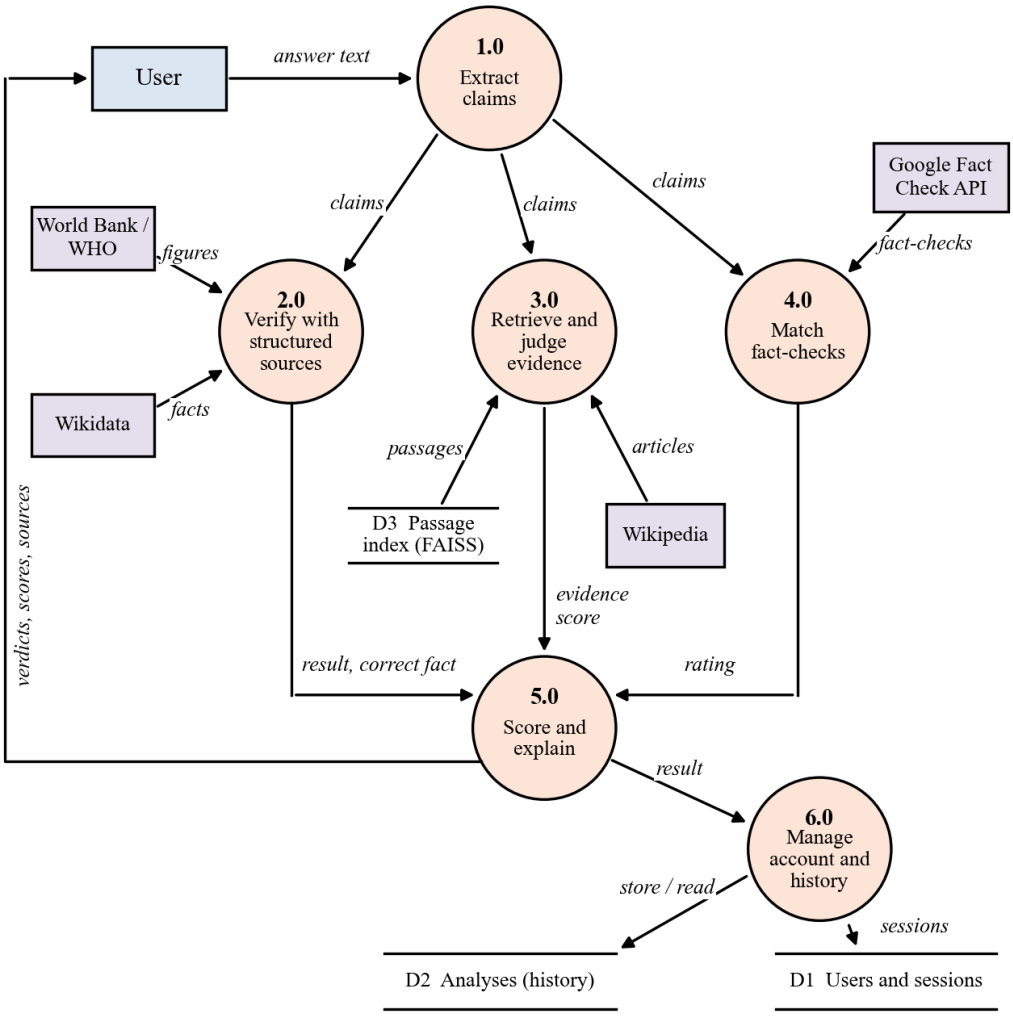


Fig. 4.3 Data flow diagram, level 1

4.3 MODULE DESIGN

The code is organised in three Python packages (models, data sources, back end) and a React application. Table 4.1 lists the main modules.

Table 4.1 Main modules

Module	Responsibility
member3_backend/claim_extractor.py	Claim extraction, pronoun resolution, filtering of non-factual sentences
member3_backend/pipeline.py	Orchestration: retrieval, inference, structured checks, fact-checks, scoring, explanations; parallel look-ups
member3_backend/evidence_rules.py	Evidence rule and on-topic test
member3_backend/factcheck_rules.py	Direction-aware matching of published fact-checks
member3_backend/crosscheck.py	Version 2 agreement rule, no-evidence filter, second-reading candidates

Module	Responsibility
member1_model/llm_verifier.py	LLM readings (Ollama, Groq and other providers), JSON parsing, answer cache, fallback between providers
member2_data/entity_linker.py, wikidata_connector.py, kg_extra.py	Entity linking (English and Hindi) and Wikidata checks
member2_data/official_stats.py	World Bank and WHO statistics with two-agency agreement
member1_model/	Embeddings, inference scorer, CMHSA scoring and weight training
member3_backend/api.py, auth.py, serve.py	Web API, accounts and security, production serving
web/src/	React and TypeScript web application

4.4 ALGORITHMS

4.4.1 Claim extraction

English text is parsed with spaCy [47]. Coordinated statements are split into separate claims and list items become claims of their own. A pronoun is replaced by the topic of the passage rather than by the nearest name; on 400 held-out Wikipedia introductions this raised the number of correctly resolved pronouns from 93 to 277 in the pre-registered test. Questions, greetings, advice and opinions are recognised and dropped. Hindi text is split into sentences with the Indic NLP Library [48], and questions, greetings and advice are dropped.

4.4.2 Evidence retrieval

Claims and passages are encoded with a multilingual XLM-R sentence encoder [42], [28] and compared in a FAISS index [43] built from 102,069 Wikipedia passages (89,594 English and 12,475 Hindi). When a claim concerns an article that is not yet indexed, the system fetches the article from Wikipedia, embeds it and adds it to the index while it runs. The ten most similar passages are retrieved, and those with a cosine similarity of at least 0.3 are passed on.

4.4.3 Inference and the evidence rule

A multilingual mDeBERTa-v3 model [44] fine-tuned on MultiNLI and XNLI [33], [34] gives, for each passage p, the probability e(p) that it entails the claim and

the probability c(p) that it contradicts it. The passages are combined by the evidence rule:

$$s = 0.5 + 0.5 \cdot \max e(p) \qquad \text{if } \max e(p) \geq 0.5$$

$$s = 0.5 + 0.5 \cdot (\max e(p) - \max c(q) \text{ over on-topic } q) \qquad \text{otherwise}$$

A passage is on-topic when the title of its article, or a form of that title, appears in the claim. Support from any retrieved passage can therefore verify a claim, but only a passage about the claim's own subject can contradict it. The rule removes the most common false alarm of the earlier version, in which a passage about a different subject contradicted the wording of the claim. It also removes many correct detections; Section 5.10 reports this cost.

Listing 4.1 Evidence rule (member3_backend/evidence_rules.py)

```
def aggregate(rows):
    """rows: (P(entail), P(contradict), on_topic) for each passage."""
    if not rows:
        return 0.5
    max_e = max(e for e, _, _ in rows)
    if max_e >= ENTAIL_WINS:                    # 0.5: support from any passage
        return 0.5 + 0.5 * max_e
    on_topic_c = [c for _, c, t in rows if t]    # only on-topic passages contradict
    return 0.5 + 0.5 * (max_e - (max(on_topic_c) if on_topic_c else 0.0))
```

4.4.4 Structured verification

- **Official statistics.** Claims about the population, GDP, life expectancy, adult literacy or infant mortality of a country are compared with World Bank [49] figures and, for life expectancy and infant mortality, also with WHO [50] figures. A claim is judged at the precision it is written to: a GDP written as "US$ 0.1 billion" is not contradicted by an official 0.066 billion. When both agencies report a value they must agree on the verdict; if they disagree, no verdict is given.
- **Knowledge graph.** Claims about places and dates of birth and death, founding dates, capitals, authorship, current office-holders, quantities and currencies are matched to Wikidata [35] through English and Hindi relation templates. When several entities share the claimed name, one of them is enough to support the claim, but a contradiction requires every namesake that has a recorded value to disagree.

Listing 4.2 Two-agency agreement for statistics (member2_data/official_stats.py)

```
def _agrees(claimed, official, tol, absolute, half=0.0):
    # within the tolerance, or within the precision the claim is written to
    return abs(claimed - official) <= max(tol if absolute else tol * official, half)


votes = {_agrees(claimed, v, tol, absolute, half) for _, _, v, _ in figures}
if len(votes) > 1:                 # World Bank and WHO disagree: no verdict
    return _unknown(kind, claimed)
status = "supported" if votes == {True} else "contradicted"
```

4.4.5 Matching published fact-checks

Fact-checks are found through the Google Fact Check Tools API [51]. A rating of “false” is carried over only if our claim entails the checked claim, and a rating of “true” only if the checked claim entails ours; neither is carried over when either direction reads as a contradiction or when the two claims are not similar enough (cosine below 0.5). This keeps a negated claim, such as “hot water does not kill the coronavirus”, from inheriting the rating of its opposite. If the applicable fact-checks disagree, none is used.

Listing 4.3 Direction-aware fact-check matching (member3_backend/factcheck_rules.py)

```
@property
def applies(self):
    """Does the fact-check's clear rating carry over to our claim?"""
    if self.similarity < FC_MIN_SIM or self.contra > FC_MAX_CONTRA:   # 0.50 / 0.20
        return False
    if self.check.verdict == "false":
        return self.entail_claim_to_check >= FC_MIN_ENTAIL            # 0.70
    if self.check.verdict == "true":
        return self.entail_check_to_claim >= FC_MIN_ENTAIL
    return False
```

4.4.6 Scoring: the Composite Multilingual Hallucination Scoring Algorithm (CMHSA)

CMHSA was designed as a weighted combination of three signals for a claim c with evidence E and knowledge graph G:

$$HalluScore(c) = \alpha \cdot SemSim(c, E) + \beta \cdot NLI(c, E) + \gamma \cdot KG(c, G), \quad \alpha + \beta + \gamma = 1$$

SemSim is the similarity of the best retrieved passage, NLI the evidence-rule score s, and KG the knowledge-graph answer. The weights are produced by a softmax, so they are non-negative and sum to one, and they are trained separately for each language with class-balanced weighting on the training claims. A signal that is missing for a claim is left out and the remaining weights are rescaled. Two design

decisions, both made during development, determine how the formula behaves in practice:

- **Similarity is excluded:** SemSim measures whether a passage is on the same topic, not whether it supports the claim, so it was excluded from training and α is fixed at exactly 0. The trained weights are $\beta = 0.500$ and $\gamma = 0.500$ for English and $\beta = 0.498$ and $\gamma = 0.502$ for Hindi. Similarity is still used, but only as the 0.3 relevance cut-off that decides which passages reach the inference model.
- **The knowledge graph decides when it answers:** if statistics or Wikidata return supported or contradicted, the score is 1.0 or 0.0 and the other signals are not mixed in. This rule was chosen from an exploratory analysis of the 40 relational claims (Section 5.9).

Since $\alpha = 0$ and a knowledge-graph answer overrides the formula, the weighted sum never actually blends two signals in the running system. The score of a claim is therefore set by the first of the following that applies:

1. Statistics or Wikidata answered: score 1.0 if supported, 0.0 if contradicted; the correct value is shown.
2. A published fact-check applies: score 0.05 if rated false, 0.95 if rated true.
3. Relevant passages were found: in Version 1 the evidence-rule score s; in Version 2 the verdict of the LLM cross-check (Section 4.4.7), with the score set to the mean of s and the LLM's probability, kept inside the range of that verdict.
4. None of these: score 0.5 (insufficient evidence) in Version 1; in Version 2 the LLM cross-check still reads the claim, without passages.

The verdict follows from the score: below 0.35 Hallucinated, above 0.65 Verified, and Uncertain in between.

Listing 4.4 Claim scoring (member1_model/cmhsa.py)

```
alpha, beta, gamma = self.get_weights(language)        # alpha = 0.0 (SemSim excluded)
raw = {"semsim": (alpha, sub_scores.sem_sim), "nli": (beta, sub_scores.nli_prob),
       "kg": (gamma, sub_scores.kg_conf)}
active = {k: (w, v) for k, (w, v) in raw.items() if v is not None}
if KG_DECIDES and "kg" in active:                       # statistics / Wikidata answered
    hallu_score = float(sub_scores.kg_conf)             # 1.0 supported, 0.0 contradicted
else:
    weight_sum = sum(w for w, _ in active.values())
    hallu_score = sum((w / weight_sum) * v for w, v in active.values())
```

4.4.7 LLM cross-check (Version 2)

The first version caught few hallucinations that no structured source covers, because the inference model accepts a passage that merely discusses a topic as support. An LLM catches many more, but in Version 1's own test an LLM judge almost quadrupled the false alarms (Section 5.10). The cross-check therefore uses the LLM as a second reader whose verdict counts only together with the evidence. The model is gpt-oss-20b [46], an open-weight mixture-of-experts model with 21 billion parameters, of which 3.6 billion are active for each word, released under the Apache 2.0 licence. It runs on the project laptop with Ollama [45], partly on the GPU and partly on the CPU; the same model can also be reached through the Groq and OpenRouter services: when Groq's free per-minute limit is reached OpenRouter answers instead, and the laptop model is the last fallback, without waiting.

For every claim that no structured source or fact-check decided, the first reading gives the model the claim and the five best passages (600 characters each) and allows it to use its own knowledge. It answers in JSON with a verdict (Verified, Uncertain or Hallucinated), a probability that the claim is true, the wrong words copied from the claim, and a one-sentence reason; decoding is greedy (temperature 0). The verdict j is combined with the evidence-rule score s as follows (Fig. 4.4, Listing 4.5):

1. If the reason of a Hallucinated j only says that the passages do not mention the fact ("no evidence", "not stated"), j counts as Uncertain: missing evidence is not proof of falsehood.
2. If $s > 0.65$ and j is Verified, the claim is Verified: the passages and the LLM agree.
3. If j is Hallucinated and the passages do not support the claim ($s \leq 0.65$), the claim is about to be flagged. A second reading first asks the same model, now restricted to all ten retrieved passages, whether they state the claim; if they do, the claim becomes Uncertain, otherwise it is Hallucinated. A passage contradiction alone never flags a claim.
4. If the claim leans true (j is not Hallucinated and either $s > 0.65$ or j is Verified), the second reading is asked whether the passages state it; if they do, the claim

becomes Verified, otherwise it stays Uncertain. Optionally (a switch that is off by default), a second reading that finds the claim false flags it.

5. Otherwise the claim is Uncertain.

The wrong words shown in the web application are the claimed value that Wikidata contradicts, or else the words the LLM named as wrong, or else the whole claim. Claims are cross-checked in parallel when a cloud provider is used, and each answer is cached, so a repeated claim costs no new LLM call.

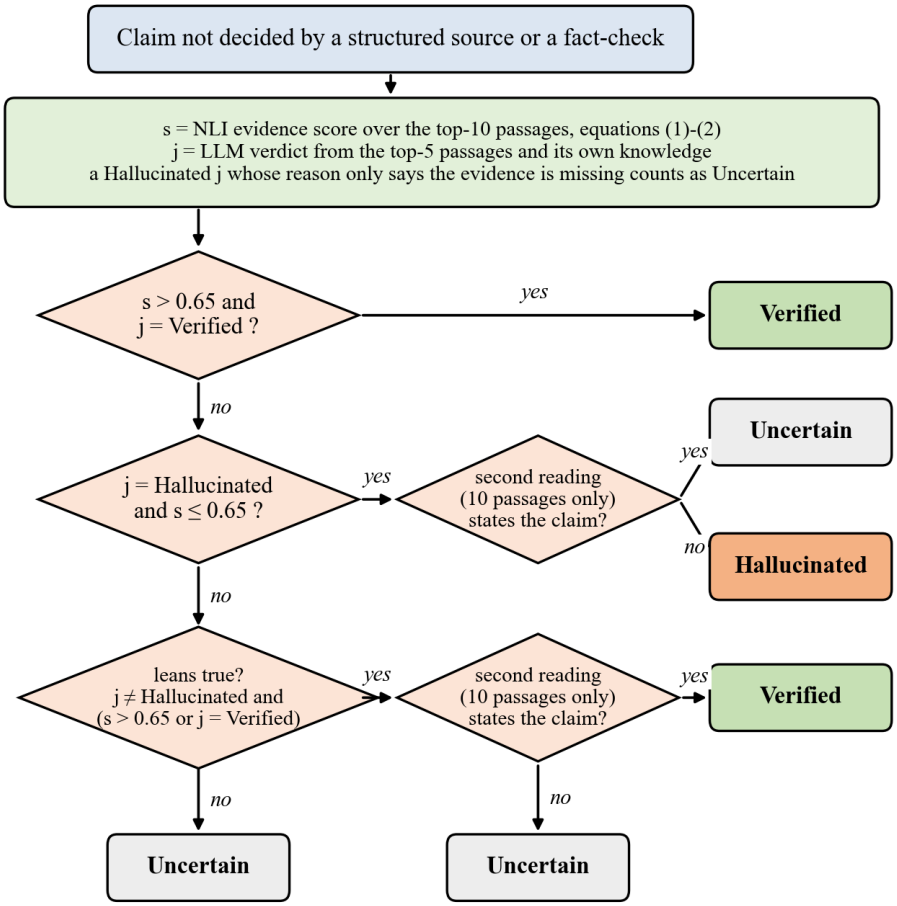


Fig. 4.4 Decision of the LLM cross-check for one claim

Listing 4.5 Agreement rule of the cross-check (member3_backend/crosscheck.py)

```
def crosscheck_verdict(v1_verdict, decided_by_source, s, llm,
                       number_missing=False, symmetric=True, flag_needs_llm=False):
    if decided_by_source or llm not in ("Verified", "Hallucinated", "Uncertain"):
        return v1_verdict              # a structured source decided, or no LLM answer
    supported = s is not None and s > SUPPORT and not number_missing      # 0.65
    contradicted = s is not None and s < CONTRADICT                       # 0.35
    if contradicted and not flag_needs_llm and not (symmetric and llm == "Verified"):
        return "Hallucinated"          # off in the final version (flag_needs_llm)
    if supported and llm == "Verified":
        return "Verified"              # both readers agree
    if not supported and llm == "Hallucinated":
        return "Hallucinated"          # then checked by the second reading
    return "Uncertain"
```

4.4.8 Whole-text verdict

The overall verdict is taken from the claim verdicts, not from an average, so that one false claim cannot be hidden among many true ones. A text is Hallucinated when at least one third of its claims are Hallucinated; otherwise it is Uncertain if any claim is Hallucinated or Uncertain; otherwise it is Verified. The overall score shown beside the verdict is the mean of the claim scores.

4.5 DATABASE DESIGN

Accounts and history are stored in SQLite. Table 4.2 lists the tables. Passwords are stored only as salted scrypt hashes, and session tokens and password-reset codes only as SHA-256 hashes, so a copy of the database cannot be used to sign in.

Table 4.2 Database tables

Table	Main columns	Purpose
users	id, email (unique), name, password_hash, created_at	Registered accounts
sessions	token_hash, user_id, created_at, expires_at	Signed-in sessions
analyses	id, user_id, created_at, language, input_text, result_json, total_claims, flagged_count	History of checks
password_resets	code_hash, user_id, expires_at	One-time password-reset codes (valid 30 minutes)

4.6 FLOWCHART OF A FACT-CHECK REQUEST

Fig. 4.5 traces a single request from the input text to the final verdict.

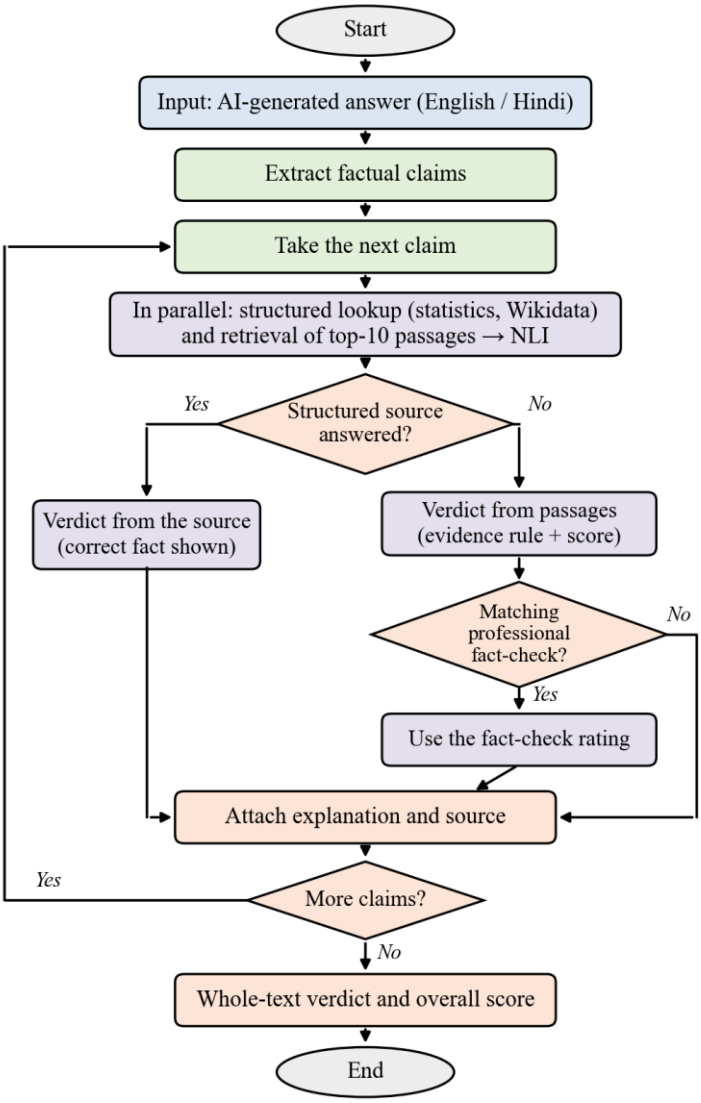


Fig. 4.5 Flowchart of a fact-check request

CHAPTER 5

IMPLEMENTATION AND RESULTS

5.1 TECHNOLOGIES USED

Table 5.1 Technologies used

Purpose	Technology
Programming	Python 3.13 (back end), TypeScript (web application)
Deep learning	PyTorch [52], Hugging Face Transformers [53], Sentence-Transformers
Models	paraphrase-xlm-r-multilingual-v1 (embeddings); mDeBERTa-v3-base-mnli-xnli (inference); spaCy en_core_web_sm (parsing)
Search	FAISS index of Wikipedia passages; live Wikipedia search
Structured data	Wikidata; World Bank Open Data; WHO Global Health Observatory; Google Fact Check Tools API
LLM (Version 2)	gpt-oss-20b [46] through Ollama [45] (local) or the Groq and OpenRouter APIs
Web	FastAPI, Uvicorn, SQLite, React, Vite, Tailwind CSS

5.2 IMPLEMENTATION OF THE MODULES

A request passes through the modules of Table 4.1 in the order of Fig. 4.1. Most of the waiting time comes from web look-ups rather than from the models, so the implementation concentrates on them. Up to six claims are looked up at the same time; each external service is paced separately, so that Wikipedia, Wikidata and the World Bank do not wait for one another; recent Wikidata and Wikipedia searches are remembered for a day; and a slow statistics service is abandoned after 8 seconds and one retry. None of these changes alters a verdict: the 13 claims of three test texts received the same verdicts before and after.

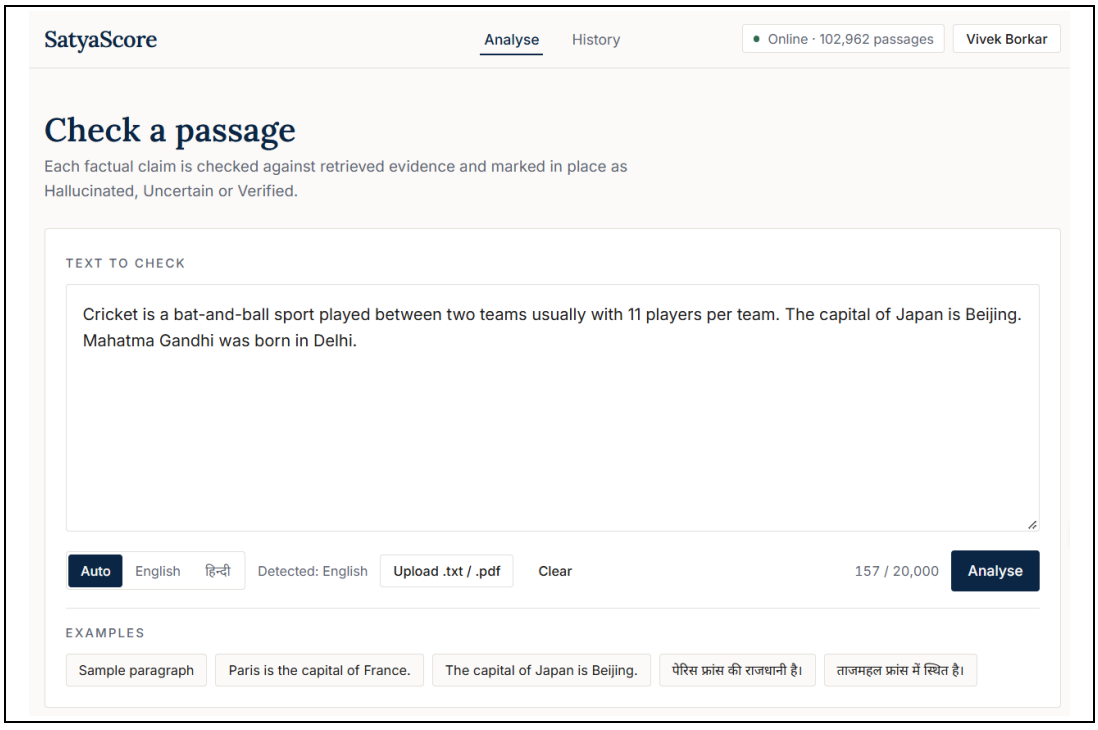
The LLM cross-check is switched on with a server setting, so that the same code runs Version 1 or Version 2; the server's health check reports which one is running. The LLM providers are tried in order: Groq first when a key is set, then OpenRouter, and the local Ollama model last; when a service answers that its limit is reached, the next one is tried at once, so a limit costs no waiting. Readings are cached by claim and passages, a provider that fails is skipped for a minute, and a reply that is

not valid JSON is read tolerantly or asked for again. If no LLM answers at all, the claim keeps its Version 1 verdict and the claim card says so.

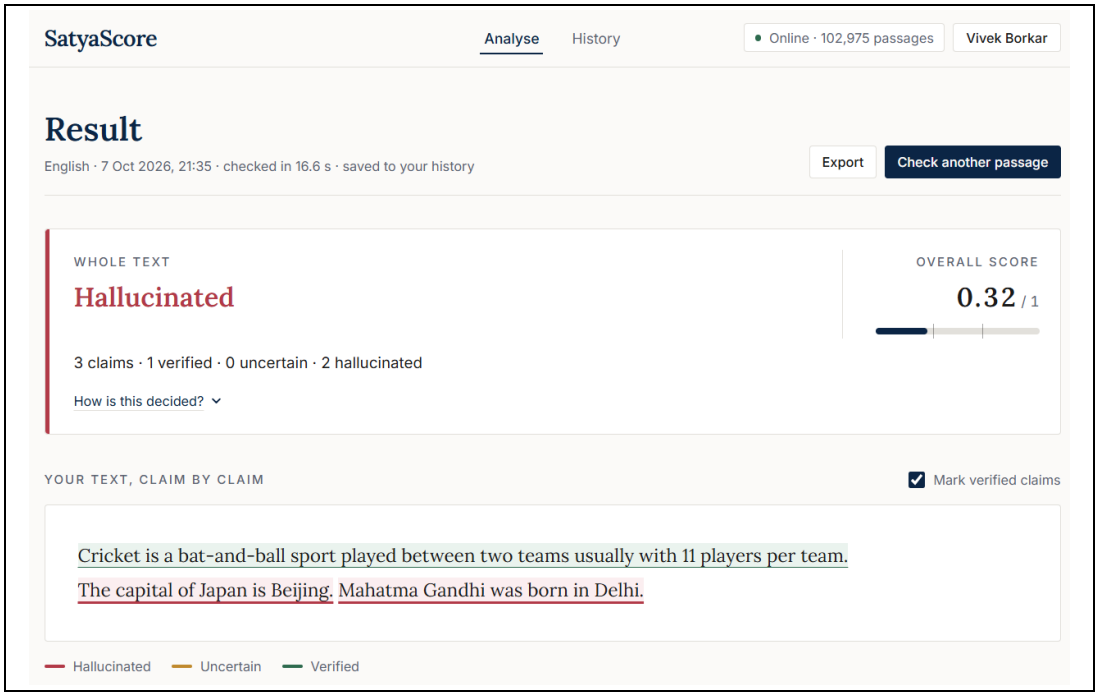
For demonstrations, a single server process (serve.py) delivers both the website and the API, and a secure public https link to the laptop is created with a Cloudflare tunnel; a script starts Version 2 with all its settings. The application is also packaged as a container for cloud hosting. The code base has 323 automated tests.

5.3 WEB APPLICATION

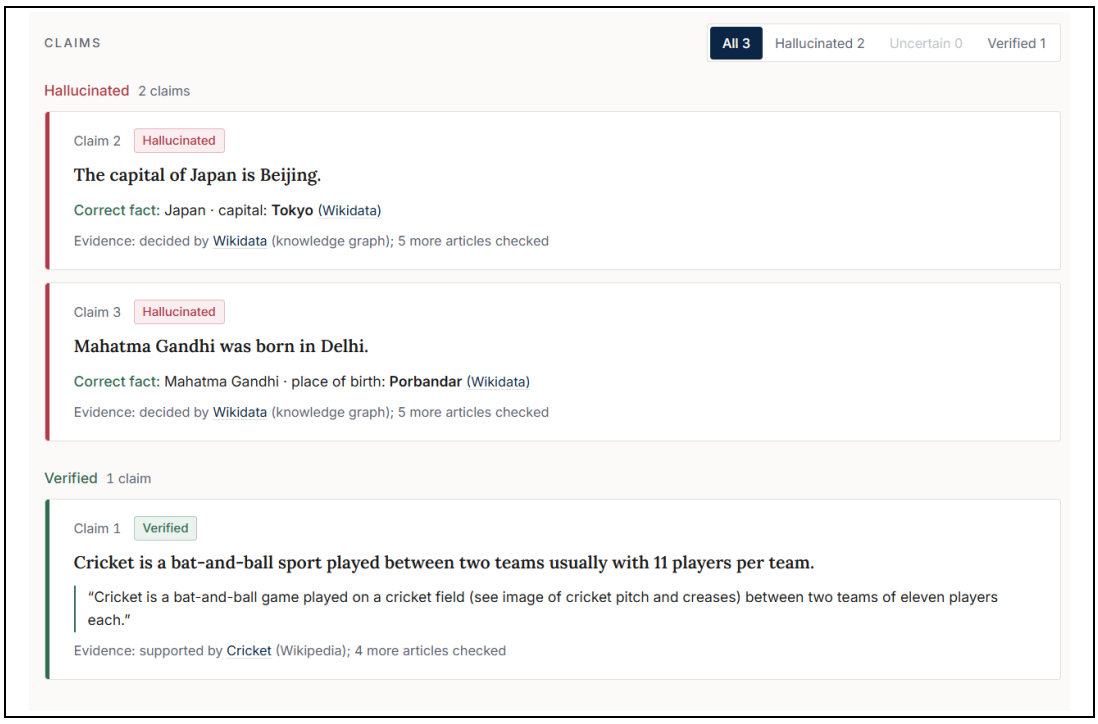
A user pastes or uploads an answer, leaves the language on automatic detection (Devanagari script means Hindi) or chooses it, and starts the check. The result page shows the whole-text verdict and overall score, the text with each claim underlined in the colour of its verdict, and a card for every claim with its source, evidence and, where available, the correct fact. Signed-in users keep a history of their checks and can export results as CSV or JSON. In Version 2 the wrong words of a flagged claim are underlined in the text, and each claim card also shows what each reader said: the passages (support, contradict or unclear, with the score), the LLM with its reason, and the second reading. Screenshots 5.1 to 5.5 were taken from the running system in Version 1, and Screenshot 5.6 in Version 2.



Screenshot 5.1 Input page with an example answer



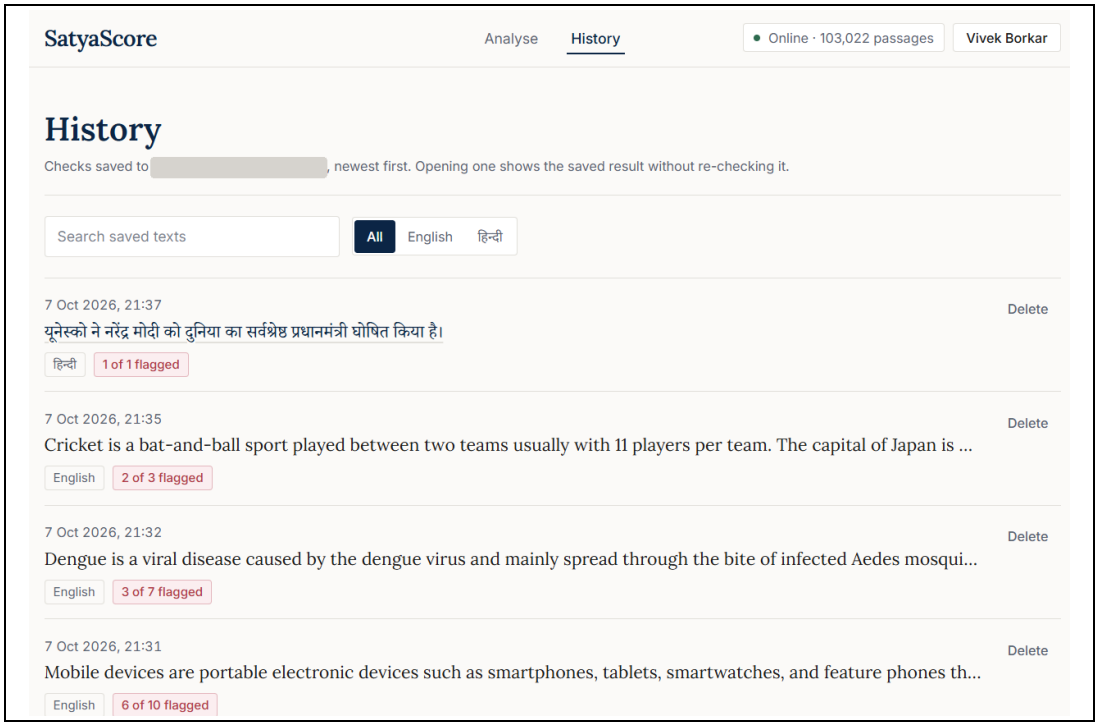
Screenshot 5.2 Results page: whole-text verdict and highlighted claims



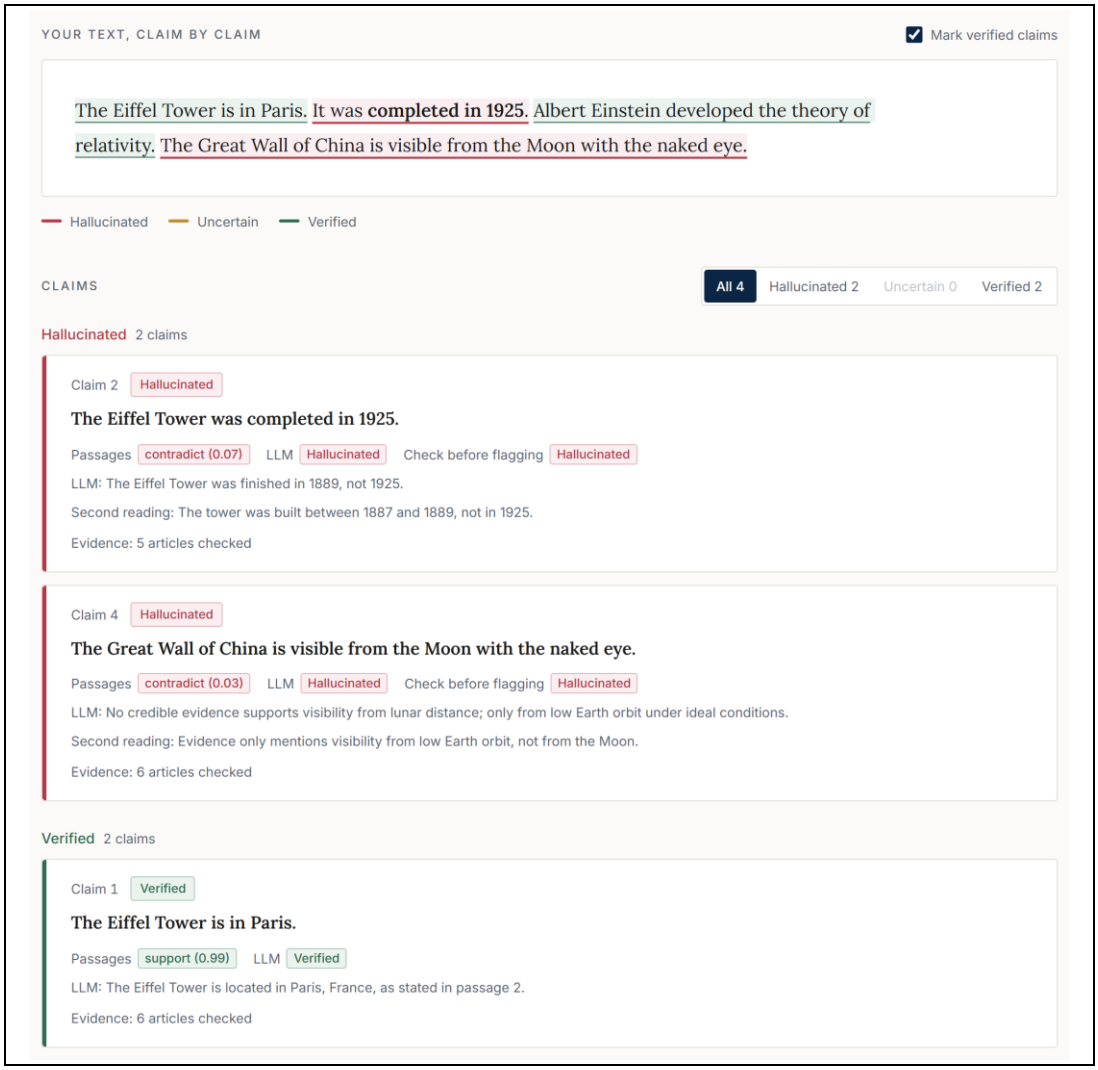
Screenshot 5.3 Claim cards with the correct fact from Wikidata



Screenshot 5.4 A Hindi claim decided by a published fact-check



Screenshot 5.5 History of saved checks (account e-mail hidden)



Screenshot 5.6 Version 2: wrong words marked and the verdict of each reader

Security measures: passwords are hashed with scrypt and a per-user salt; sessions use HttpOnly cookies; state-changing requests from other websites are refused by an origin check; failed sign-ins are throttled (five failures for the same account from the same address within 15 minutes block further attempts until the window clears); checks are rate-limited per user and per network address; a password change requires the current password; and a forgotten password is reset with a one-time code that is valid for 30 minutes.

5.4 DATASETS AND ANNOTATION

Table 5.2 lists the data used for evaluation. The main set consists of claims extracted from answers that the Llama 3.3 (70B) model [2] gave to questions in five domains: government, healthcare, science, sports and history.

Table 5.2 Datasets

Dataset	Source	Size	Used for
Labelled chatbot claims	Llama 3.3 (70B) answers, five domains	456 claims (266 English, 190 Hindi); 418 well-formed	Weight training and main evaluation
Train / test split	418 well-formed claims, grouped by source question; test fold fixed in advance	334 / 84 claims; binary sets 303 (258 true, 45 hallucinated) / 76 (66 true, 10 hallucinated)	Main evaluation
Relational claims	Answers from gpt-oss-120b and qwen3.8-27b	45 labelled by two annotators; 40 where they agreed (25 true, 15 hallucinated); English	Knowledge-graph study
Unseen true sentences	Wikipedia sentences not used in development	99 claims and 128 claims (two samples)	False-alarm tests
Statistics test	True and altered figures for 40 randomly chosen countries	137 true and 137 altered claims	Statistics check
Fact-check test	Known rumours and true statements	40 claims (25 rumours, 15 true)	Fact-check matching
Pronoun test	Held-out Wikipedia introductions	400 introductions	Pronoun resolution
Common-sense set	Statements written for the error analysis	15 statements (9 false)	Error analysis
FEVER (public) [6]	paper_dev split, random sample (seed 20261010)	150 supported and 150 refuted claims (English)	Public benchmark
Mu-SHROOM (public) [22]	SemEval-2025 Task 3 test sets	150 Hindi and 154 English model answers with annotated spans	Public benchmark
Version 2 sets (public and constructed)	FEVER; Mu-SHROOM claims (English, Hindi); TruthfulQA [9]; X-FACT Hindi rumours [21]; Hindi Wikipedia sentences with one number changed, as written and reworded	Development and locked test splits per set	Choosing and evaluating the cross-check
Version 2 held-out sample	Locked test splits, up to 60 true and 60 false items per set and language, none used before	620 items (246 true, 374 false)	Final comparison of Version 1 and Version 2

Annotation was done in two ways, and the difference matters. The 456 main claims were labelled by a team of four annotators, the authors, who shared the claims among themselves so that each claim was labelled by one of them. An AI system

suggested a verdict for every claim, and the annotator confirmed or overrode each suggestion. Overall, 79% of the final labels agree with the suggestion. Because the suggestion was visible, these labels may be biased towards it. Two blind checks measured this. In the first, two annotators independently labelled 56 claims without seeing any suggestion; their agreement on "hallucination or not" was $\kappa = 0.53$ (88% raw agreement). In the second, 50 claims were labelled blind; where the two annotators agreed (46 claims), the existing label matched theirs in 45 cases. The relational claims were labelled by two annotators with AI assistance, and only the 40 claims on which both agreed are used.

The Version 2 sets contain no label written by the project team. A MuSHROOM claim is labelled false when it overlaps a human-marked hallucinated span by at least three characters, and claims with one or two overlapping characters are dropped. TruthfulQA contributes the best answer and the first incorrect answer to each question, stated without the question. X-FACT contributes Hindi statements rated false by professional fact-checkers. The Hindi Wikipedia true items are sentences taken from articles, and each false item is the same sentence with one number changed; the reworded set paraphrases the true sentences with a different LLM (qwen3.8-27b), keeps a paraphrase only if the inference model finds that each sentence entails the other with probability at least 0.80 and all numbers are kept, and changes one number for the false item. The chatbot claims of the first version were used while the cross-check was developed, so they are not used to report it.

5.5 EVALUATION METHOD

For each proposed change, a decision rule stating what the change had to achieve (for example, fewer false alarms without fewer correct verdicts) was written down and committed to the project repository before the change was run. Most evaluations followed this protocol. Two analyses of how the knowledge graph should be used were exploratory: they were made after the labels had been seen and are marked as such. In four cases the final decision went against the outcome of the rule; these are reported in Section 5.10.

Version 2 followed a stricter protocol, recorded in the project repository before each run. The LLM was chosen in a bake-off on 202 development items fixed in advance: gpt-oss-20b reached the highest balanced accuracy under the cross-check

(57%, with 7% false alarms), ahead of gpt-oss-120b (56%, 10%); qwen3.8-27b answered fewer than 90% of the items and was excluded by rule. Held-out samples were drawn from the locked test splits, no item twice, and each was run once for both versions. Before each run the configuration and its acceptance criterion were written down; the final criterion required, relative to Version 1, at least twice as many catches, at most half as many false items passed as Verified, false alarms at most 3 points higher, and true items Verified at most 5 points lower. Paired differences are tested with exact McNemar tests, and the ratio of catches with a bootstrap over items (2,000 resamples).

The measures are defined as follows. A false alarm is a true claim marked Hallucinated. A hallucination is caught when it is marked Hallucinated. A verdict is correct when it matches the label; Uncertain is never counted as correct. Claims from the same chatbot answer are always on the same side of the train/test split, so that the context of an answer cannot leak into the test.

5.6 RESULTS ON REAL CHATBOT CLAIMS AND UNSEEN DATA

Table 5.3 Results of the running system on chatbot claims and unseen data

Set	Correct	False alarms	Hallucinations caught	Hallucinations marked Verified	Uncertain
Training claims (303)	157	9 of 258	5 of 45	27 of 45	110
Test claims (76)	38	3 of 66	0 of 10	8 of 10	27
Unseen true sentences (128)	113 (Verified)	4 of 128	—	—	11

The system seldom calls a true statement false: 4 of 128 unseen true sentences and 3 of 66 true test claims. It is, however, weak at catching hallucinations that no structured source covers: on the training claims 27 of 45 hallucinations were marked Verified and only 5 were caught, and on the small test set none of 10 was caught. Many claims end as Uncertain.

5.7 RESULTS ON PUBLIC BENCHMARKS

To allow comparison with other work, the unchanged Version 1 system was run on two public, human-labelled benchmarks. The protocol (samples, measures,

baselines) was written down before the run, and nothing in the system was tuned on these data.

FEVER [6]. Table 5.4 gives the results on 300 claims from the FEVER paper_dev split. Of the supported claims 71% were verified and 17 of 150 raised false alarms; of the refuted claims 43% were caught. The system gave a definite verdict on 71% of the claims, and 81% of those verdicts were correct. Statistics or Wikidata decided only 9 claims (7 correctly) and no fact-check applied, so the inference-only figures are almost the same. Among the 17 false alarms are facts that were true in 2017 but have changed since ("Marillyn Hewson is currently the President of Lockheed Martin"), vague statements ("Kesha has a full name"), genuine inference errors ("Tim McGraw acted in The Blind Side") and one knowledge-graph error ("The Bee Gees wrote songs").

Table 5.4 FEVER sample: live system and inference only

	Live system	**Inference only**
Supported claims marked Verified	107 of 150	107 of 150
Supported claims marked Hallucinated (false alarms)	17 of 150	17 of 150
Refuted claims marked Hallucinated (caught)	64 of 150	64 of 150
Refuted claims marked Verified (missed)	24 of 150	23 of 150
Accuracy of definite verdicts	81%	81%

Mu-SHROOM [22]. Each model answer was checked as a whole, and the characters of the claims marked Hallucinated were compared with the annotated hallucinated spans (Table 5.5). The system flagged at least one claim in 7% of the Hindi and 30% of the English answers. When it flagged a claim, the claim lay on an annotated span in 71% (Hindi) and 82% (English) of cases, but because it flags so little, its span IoU is far below a baseline that marks every character and far below the teams of the shared task (best IoU 0.74 for Hindi and 0.65 for English; team means 0.50 and 0.40). No Hindi claim and only 3 English claims were decided by a structured source.

Table 5.5 Mu-SHROOM test sets: span IoU and correlation

Method	**Hindi (150): IoU / correlation**	**English (154): IoU / correlation**
Satya-Score (live)	0.011 / 0.053	0.102 / 0.112

Method	Hindi (150): IoU / correlation	English (154): IoU / correlation
Mark-all baseline	0.271 / 0.000	0.349 / 0.000
Mark-none baseline	0.000 / 0.000	0.033 / 0.000
Best team (task report)	0.74 / 0.78	0.65 / 0.62
Mean of teams (task report)	0.50 / 0.52	0.40 / 0.37

The benchmarks confirm the picture from the project's own data: when the system flags a claim it is usually right, but it flags little, and the structured sources that make it reliable apply to few of the claims in these benchmarks. This is the gap the LLM cross-check was built to close.

5.8 RESULTS OF THE LLM CROSS-CHECK (VERSION 2)

Table 5.6 gives the final comparison of the two versions on the 620 held-out items, run once. With the cross-check the system caught 176 of 374 false items instead of 95 (McNemar test: 92 newly caught, 11 lost, $p < 0.001$) and passed 23 of them as Verified instead of 83 ($p < 0.001$). Of the 246 true items, 139 were Verified instead of 137 and 32 were flagged instead of 28; neither change is significant ($p = 0.87$ and 0.63; difference in false alarms +1.6 points, 95% interval −3.2 to +6.6). Three of the four criteria were met. Catches rose by a factor of 1.85 (95% interval 1.61 to 2.19), short of the required 2, so by its own rule Version 2 was not made the default; it is offered as a mode that can be switched on.

Table 5.6 Held-out comparison of Version 1 and Version 2 (620 items)

Measure	Version 1	Version 2	Criterion fixed in advance
False items caught	95 of 374 (25%)	176 of 374 (47%)	At least 2× Version 1: not met (1.85×)
False items passed as Verified	83 of 374 (22%)	23 of 374 (6%)	At most half: met
True items Verified	137 of 246 (56%)	139 of 246 (57%)	At most 5 points lower: met
False alarms on true items	28 of 246 (11%)	32 of 246 (13%)	At most 3 points higher: met

Table 5.7 breaks the result down by set. The gains are largest where Version 1 relied on weak text evidence: Hindi Wikipedia sentences with a changed number (caught 3% → 39%), false Mu-SHROOM Hindi claims (7% → 45%) and TruthfulQA

misconceptions (17% → 43%). On FEVER the cross-check caught more and also flagged fewer true claims, although it Verified slightly fewer (72% → 67%). The costs fall on TruthfulQA, where false alarms rose from 18% to 27% because many answers are vague without their question, and on the reworded Hindi sentences (3% → 11%). X-FACT rumours were already caught by published fact-checks; without that source Version 1 caught only 5% of them.

Table 5.7 Results per set, Version 1 → Version 2 (percent of items)

Set (true / false items)	Caught	Passed as Verified	False alarms
FEVER, English (60 / 60)	37 → 62	20 → 5	13 → 7
Hindi Wikipedia (38 / 38)	3 → 39	13 → 5	0 → 0
Hindi Wikipedia, reworded (36 / 36)	28 → 47	33 → 0	3 → 11
Mu-SHROOM, English (52 / 60)	13 → 20	23 → 15	15 → 15
Mu-SHROOM, Hindi (0 / 60)	7 → 45	38 → 5	—
TruthfulQA, English (60 / 60)	17 → 43	27 → 8	18 → 27
X-FACT, Hindi (0 / 60)	67 → 70	2 → 2	—

Table 5.8 removes the parts of the rule one at a time, using the LLM answers already stored for the same 620 items; this analysis was made after the result above and is exploratory. The LLM alone caught most false items but flagged 28% of true ones. Requiring agreement with the passages lowered false alarms to 22%, the no-evidence filter to 16% at the cost of catches, the second reading brought true claims Verified back from 47% to 57%, and requiring the LLM to say false, with the passages-only check before flagging, brought false alarms to 13%. The optional switch that lets a second reading's "Hallucinated" flag a claim raised catches to 49% and false alarms to 15%.

Table 5.8 Ablation on the held-out sample (exploratory)

Variant	Caught	Passed as Verified	True Verified	False alarms
Version 1 (no LLM)	25%	22%	56%	11%
LLM alone	86%	11%	65%	28%
Agreement rule only	68%	4%	47%	22%
+ no-evidence filter	49%	4%	47%	16%
+ second reading	49%	6%	57%	16%

Variant	Caught	Passed as Verified	True Verified	False alarms
+ LLM must say false, check before flagging (Version 2)	47%	6%	57%	13%
Version 2 + second reading may flag (optional)	49%	6%	57%	15%

The final rule is the fourth version tested on held-out data, each time on a fresh sample (Table 5.9). The plain agreement rule failed the first criterion, an absolute limit of 5% false alarms; inspection showed the LLM calling claims false only because the passages did not mention them. That limit had been set when Version 1's own rate on chatbot claims looked like 3–6%, whereas on these sets it is 9–12%, so before the next run the criterion was changed to one relative to Version 1 and the no-evidence filter was added. The criteria and each change were written down before the sample they were tested on.

Table 5.9 Development rounds of the cross-check on held-out samples (Version 2 against Version 1)

Round (items)	Change	Caught	False alarms	Outcome
1 (201)	Agreement rule	58% vs 20%	17% vs 9%	Failed (false alarms)
2 (175)	+ no-evidence filter	47% vs 15%	12% vs 12%	Passed; true Verified 40% vs 56%
3 (162)	+ second reading	37% vs 19%	14% vs 5%	Failed (catches, false alarms)
4 (620)	+ check before flagging	47% vs 25%	13% vs 11%	3 of 4 criteria met

Marking only the wrong words instead of whole claims also improved span overlap on Mu-SHROOM (Table 5.10, answers in the held-out sample), especially for Hindi, although the system remains below a baseline that marks every character and far below the shared-task teams, which were built for span marking.

Table 5.10 Mu-SHROOM span IoU on the held-out answers

Method	Hindi (52 answers)	English (72 answers)
Version 1, flagged claims	0.010	0.114
Version 2, flagged claims	0.097	0.172
Version 2, wrong words only	0.189	0.184

Method	Hindi (52 answers)	English (72 answers)
Mark-all baseline	0.318	0.307

5.9 RESULTS OF THE STRUCTURED SOURCES

Table 5.11 Results of the structured sources

Component	Test data	Result
Knowledge graph, pre-registered run	40 relational claims (English)	11 of 13 decisions correct: supported 8 of 8, contradicted 3 of 5
Knowledge graph after a place-linking fix (exploratory)	Same 40 claims	12 of 12 decisions correct: supported 10 of 10, contradicted 2 of 2
Knowledge graph ablation, pre-registered	Same 40 claims	Inference model alone against inference plus knowledge graph: AUC 0.704 → 0.784 (difference +0.080, 95% interval +0.003 to +0.189); Hallucinated-F1 0.512 → 0.615
Knowledge graph deciding when it answers (after the fix)	Same 40 claims	Correct verdicts: 14 with the inference model alone, 24 when the knowledge graph decides
Official statistics	40 randomly chosen countries	137 of 137 altered figures contradicted; of 137 true figures, 131 Verified, 0 contradicted, 6 without an answer
Published fact-checks	40 claims (25 rumours, 15 true)	No wrong decision; rumours caught 3 → 10 of 25; correct verdicts 13 → 20 of 40
Pronoun resolution	400 Wikipedia introductions	Correctly resolved 93 → 277; wrongly resolved 206 → 128

The two incorrect decisions of the pre-registered knowledge-graph run came from place names linked to the wrong Wikidata items (for example, a town linked to a given name and a village to a monorail station). The linking was corrected after these results had been seen, which is why the 12-of-12 figure is exploratory. The rule that the knowledge graph decides when it answers was also chosen on these 40 claims after the labels were known: its score separated true from hallucinated claims slightly better than the weighted combination (AUC 0.872 against 0.848; difference +0.024, 95% interval −0.000 to +0.083). The pre-registered ablation, by contrast, was fixed before any label existed and shows that adding the knowledge graph to the inference signal improves the separation of true and hallucinated claims. The knowledge-graph

results are for English claims; the Hindi templates are implemented but were not part of this test.

5.10 CHANGES TESTED AND NOT ADOPTED

Table 5.12 lists the main changes to Version 1 and the decision taken on each. The first four rows are the cases in which the final decision went against the outcome of the rule. The changes of Version 2 are reported in Section 5.8.

Table 5.12 Changes tested and the decisions taken

Change	Measured outcome	Decision
Evidence rule (Section 4.4.3)	False alarms on 99 unseen true statements 33 → 8; but its pre-fixed check on training AUC failed, and hallucinations caught fell (train 11 → 3 of 45, test 3 → 0 of 10, relational 12 → 2 of 15)	Adopted against the rule, to avoid calling true text false
Reading 10 passages instead of 3	Correct training verdicts 130 → 170 of 303; but its pre-fixed check on false alarms failed (8 → 11 of 258)	Adopted against the rule
LLM judge (open mode)	Met its rule (training AUC 0.717 against 0.552); caught 18 instead of 5 of 45 hallucinations, but false alarms rose from 9 to 35 of 258	Kept off against the rule; Version 2 uses the LLM as a cross-checker instead
Larger inference models	The rule picked other models (English: train AUC 0.602 against 0.539, intervals overlapping; Hindi: too few hallucinations to judge)	Current model kept against the rule
Number guard (two versions)	Removed correct verifications (115 → 101; 109 → 103)	Not adopted
MedlinePlus as a second source	False alarms 9 → 11 of 258	Not adopted
Strict support rule	True claims Verified 152 → 47; false alarms 9 → 11	Not adopted
Wikipedia articles fetched on demand	First test: no gain (ΔAUC −0.020); second test, with the evidence rule: all three conditions met	Adopted after the second test
MiniCheck grounding checker (probe)	Still supported 7 of 9 false common-sense statements, as the current model did	Not pursued

These results are informative in themselves. The method that caught the most hallucinations, an LLM judge, did so at a cost in false alarms that the project was not willing to accept, and the evidence rule bought its low false-alarm rate with fewer detections. Version 2 grew out of these two observations.

5.11 PERFORMANCE

Table 5.13 Performance on the laptop (RTX 4050 GPU)

Text and configuration	First check	Repeated check
Version 1 before optimisation: chatbot answer, 6 claims (English)	27.7 s	10.1 s
Version 1 after optimisation: same answer	5.1 s	0.7 s
Version 1: general facts, 4 claims (English)	1.6 s	0.3 s
Version 1: Hindi answer, 3 claims	3.7 s	0.4 s
Version 2, local LLM only: 6 claims (English)	45.7 s	under 3 s
Version 2, Groq first: 5 claims (English)	18.2 s	2.7 s

Without a GPU, the six-claim answer took 13.7 s on first check and 0.7 s on repetition in Version 1. Starting the server loads the models once, which takes about 20 seconds; the local LLM is loaded on first use, which adds about half a minute to the first check. In Version 2 most of the time is spent by the LLM: the 13 GB model does not fit in the 6 GB GPU memory, so Ollama runs most of it on the CPU at about 4 seconds per reading. Allowing four parallel requests to the local model made a six-claim answer slower (202 s), so it reads one claim at a time. With Groq answering first, the readings take about a second each and run in parallel, but the free tier allows only about 8,000 tokens per minute, three to four readings, so a busy minute falls back to the next service. With OpenRouter as the second service, a three-claim check took 6 to 8 s, against 41 s when the busy minutes fell back to the laptop model.

5.12 DISCUSSION AND LIMITATIONS

Comparison with existing methods. The systems in Table 2.1 were evaluated on other data and languages, so their published scores cannot be compared directly with these results, except on Mu-SHROOM (Section 5.7), where the system is far behind the shared-task teams on span-level IoU. Qualitatively, Satya-Score is the only one among them that checks Hindi claims against structured sources and states the correct fact. On this project's data, a purpose-built grounding checker and larger inference models did not improve on the model in use, and an LLM judge traded many more false alarms for its extra detections. Used as a cross-checker instead, the same kind of model nearly doubled the catches of Version 1 at a cost of about two points in false alarms.

What the cross-check shows. An LLM is a strong but unreliable witness: alone it caught 86% of the false items and flagged 28% of the true ones, so its value lies in being combined with evidence rather than in replacing it. Most of the improvement comes from claims that no structured source covers, especially in Hindi, where the inference model had found almost nothing. Each safeguard traded some catches for fewer false alarms, and the final rule sits close to Version 1's false-alarm rate. When the passages and the LLM disagree, the claim is left Uncertain rather than given either reader's verdict, which is why some false claims are reported as Uncertain instead of Hallucinated.

Error analysis. Of the 45 hallucinations in the training split, about 31 turned out to come from an earlier version of the claim extractor rather than from the chatbot. Most of the remaining ones are misconceptions or over-generalisations that no structured source records; several were marked Verified because a retrieved passage was merely about the same topic, the word-overlap weakness described by McCoy et al. [17]. On 15 common-sense statements written for this analysis, all 9 false ones were marked Verified.

Limitations:

- In Version 1, hallucinations outside the coverage of structured sources are usually missed; in Version 2 about half of the false items are still not caught.
- The Version 2 criterion of doubling the catches was missed (1.85×); the criterion was changed once after a failed run, and the final rule was chosen on samples that had already been used. All held-out samples are now used up.
- The constructed Hindi false items change a single number and are easier than real errors; Mu-SHROOM claim labels come from span overlap; TruthfulQA answers lack their question; each set contributes at most 60 true and 60 false items.
- The LLM's knowledge has a cut-off date, it can share the misconceptions of the chatbots it checks, and on the laptop it makes a check take tens of seconds.
- The labelled chatbot data are small (456 main and 40 relational claims) and contain few Hindi hallucinations (9).

- Each main claim was labelled once, by one of the four annotators, with the AI suggestion visible; agreement between independent annotators was moderate ($\kappa = 0.53$).
- The knowledge-graph rules were chosen in exploratory analyses on the same 40 claims on which they were measured.
- Live look-ups need internet access, and the WHO data may be used for non-commercial purposes only.

CHAPTER 6

CONCLUSIONS AND FUTURE SCOPE

6.1 CONCLUSION

Satya-Score checks English and Hindi answers one claim at a time and supports every verdict with a source that the reader can open. Sending each claim first to official statistics, the Wikidata knowledge graph or a published fact-check made the system precise wherever such a source applies, and the conservative evidence rule kept false alarms on true text low. The first version showed the price of that caution: most hallucinations that no structured source covers were not caught. The second version adds an open-weight LLM as a second reader whose verdict counts only when the evidence agrees. On 620 held-out items it nearly doubled the hallucinations caught (47% against 25%) and cut those passed as Verified from 22% to 6%, while false alarms rose only from 11% to 13%; it met three of the four criteria fixed in advance. Testing each change against a rule fixed in advance made clear which attractive ideas did not help, and the cases in which the project decided against a rule are reported openly. In one sentence, the contribution is an explainable bilingual claim checker that combines published fact-checks, official statistics, a knowledge graph, encyclopaedic text and an LLM that can never decide alone.

6.2 FUTURE SCOPE

- Raising the catches of the cross-check further, to the doubling we aimed for, without adding false alarms; for example by letting an LLM propose suspect claims that a structured source must then confirm.
- A faster deployment of the cross-check, for example a hosted service for the same open model.
- Extending the system to Marathi with MuRIL, IndicSBERT and IndicXNLI.
- A larger, multi-domain Hindi evaluation set, labelled blind by at least two annotators.
- Further official sources, such as Indian government open-data portals.

REFERENCES

[1] OpenAI et al., "GPT-4 technical report," arXiv:2303.08774, 2023, doi: 10.48550/arXiv.2303.08774.

[2] A. Grattafiori et al., "The Llama 3 herd of models," arXiv:2407.21783, 2024, doi: 10.48550/arXiv.2407.21783.

[3] Z. Ji et al., "Survey of hallucination in natural language generation," ACM Comput. Surv., vol. 55, no. 12, 2023, doi: 10.1145/3571730.

[4] L. Huang et al., "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," ACM Trans. Inf. Syst., vol. 43, no. 2, 2025, doi: 10.1145/3703155.

[5] S. Min et al., "FActScore: Fine-grained atomic evaluation of factual precision in long form text generation," in Proc. EMNLP, 2023, doi: 10.48550/arXiv.2305.14251.

[6] J. Thorne, A. Vlachos, C. Christodoulopoulos, and A. Mittal, "FEVER: A large-scale dataset for fact extraction and VERification," in Proc. NAACL-HLT, 2018, doi: 10.18653/v1/N18-1074.

[7] O. Honovich et al., "TRUE: Re-evaluating factual consistency evaluation," in Proc. NAACL-HLT, 2022, doi: 10.18653/v1/2022.naacl-main.287.

[8] Y. Zha, Y. Yang, R. Li, and Z. Hu, "AlignScore: Evaluating factual consistency with a unified alignment function," in Proc. ACL, 2023, doi: 10.18653/v1/2023.acl-long.634.

[9] S. Lin, J. Hilton, and O. Evans, "TruthfulQA: Measuring how models mimic human falsehoods," in Proc. ACL, 2022, doi: 10.18653/v1/2022.acl-long.229.

[10] J. Li, X. Cheng, X. Zhao, J.-Y. Nie, and J.-R. Wen, "HaluEval: A large-scale hallucination evaluation benchmark for large language models," in Proc. EMNLP, 2023, doi: 10.18653/v1/2023.emnlp-main.397.

[11] J. Wei et al., "Long-form factuality in large language models," arXiv:2403.18802, 2024, doi: 10.48550/arXiv.2403.18802.

[12] P. Manakul, A. Liusie, and M. J. F. Gales, "SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models," in Proc. EMNLP, 2023, doi: 10.48550/arXiv.2303.08896.

[13] X. Hu et al., "Knowledge-centric hallucination detection," in Proc. EMNLP, 2024, doi: 10.18653/v1/2024.emnlp-main.395.

[14] Z. Ji et al., "ANAH: Analytical annotation of hallucinations in large language models," in Proc. ACL, 2024, doi: 10.18653/v1/2024.acl-long.442.

[15] A. Mishra, A. Asai, V. Balachandran, Y. Wang, G. Neubig, Y. Tsvetkov, and H. Hajishirzi, "Fine-grained hallucination detection and editing for language models," in Proc. COLM, 2024, doi: 10.48550/arXiv.2401.06855.

[16] W. Su, C. Wang, Q. Ai, Y. Hu, Z. Wu, Y. Zhou, and Y. Liu, "Unsupervised real-time hallucination detection based on the internal states of large language models," in Findings of ACL, 2024, doi: 10.18653/v1/2024.findings-acl.854.

[17] R. T. McCoy, E. Pavlick, and T. Linzen, "Right for the wrong reasons: Diagnosing syntactic heuristics in natural language inference," in Proc. ACL, 2019, doi: 10.18653/v1/P19-1334.

[18] L. Zheng et al., "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in Proc. NeurIPS (Datasets and Benchmarks Track), 2023.

[19] L. Tang, P. Laban, and G. Durrett, "MiniCheck: Efficient fact-checking of LLMs on grounding documents," in Proc. EMNLP, 2024, doi: 10.48550/arXiv.2404.10774.

[20] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Proc. NeurIPS, 2020, doi: 10.48550/arXiv.2005.11401.

[21] A. Gupta and V. Srikumar, "X-FACT: A new benchmark dataset for multilingual fact checking," in Proc. ACL-IJCNLP (Short Papers), 2021, pp. 675–682, doi: 10.18653/v1/2021.acl-short.86.

[22] R. Vázquez et al., "SemEval-2025 Task 3: Mu-SHROOM, the multilingual shared task on hallucinations and related observable overgeneration mistakes," in Proc. SemEval, 2025, arXiv:2504.11975, doi: 10.48550/arXiv.2504.11975.

[23] B. Hikal, A. Nasreldin, and A. Hamdi, "MSA at SemEval-2025 Task 3: High quality weak labeling and LLM ensemble verification for multilingual hallucination detection," in Proc. SemEval, 2025. [Online]. Available: https://aclanthology.org/2025.semeval-1.131/

[24] M. Bala et al., "HausaNLP at SemEval-2025 Task 3: Towards a fine-grained model-aware hallucination detection," in Proc. SemEval, 2025. [Online]. Available: https://aclanthology.org/2025.semeval-1.227/

[25] P. Elchafei and M. Abu-Elkheir, "Hallucination Detectives at SemEval-2025 Task 3: Span-level hallucination detection for LLM-generated answers," in Proc. SemEval, 2025. [Online]. Available: https://aclanthology.org/2025.semeval-1.84/

[26] A. Sinha et al., "SHROOM-CAP: Shared task on hallucinations and related observable overgeneration mistakes in crosslingual analyses of publications," in Proc. CHOMPS, 2025, doi: 10.18653/v1/2025.chomps-main.7.

[27] H. Terdalkar, K. Bhojani, A. Dongare, and O. A. Behera, "BHRAM-IL: A benchmark for hallucination recognition and assessment in multiple Indian languages," in Proc. BHASHA Workshop, 2025, doi: 10.18653/v1/2025.bhasha-1.9.

[28] A. Conneau et al., "Unsupervised cross-lingual representation learning at scale," in Proc. ACL, 2020, doi: 10.18653/v1/2020.acl-main.747.

[29] S. Khanuja et al., "MuRIL: Multilingual representations for Indian languages," arXiv:2103.10730, 2021, doi: 10.48550/arXiv.2103.10730.

[30] D. Kakwani et al., "IndicNLPSuite: Monolingual corpora, evaluation benchmarks and pre-trained multilingual language models for Indian languages," in Findings of EMNLP, 2020, doi: 10.18653/v1/2020.findings-emnlp.445.

[31] S. Deode et al., "L3Cube-IndicSBERT: A simple approach for learning cross-lingual sentence representations using multilingual BERT," arXiv:2304.11434, 2023, doi: 10.48550/arXiv.2304.11434.

[32] D. Aggarwal, V. Gupta, and A. Kunchukuttan, "IndicXNLI: Evaluating multilingual inference for Indian languages," arXiv:2204.08776, 2022, doi: 10.48550/arXiv.2204.08776.

[33] A. Williams, N. Nangia, and S. R. Bowman, "A broad-coverage challenge corpus for sentence understanding through inference," in Proc. NAACL-HLT, 2018, doi: 10.18653/v1/N18-1101.

[34] A. Conneau et al., "XNLI: Evaluating cross-lingual sentence representations," in Proc. EMNLP, 2018, doi: 10.18653/v1/D18-1269.

[35] D. Vrandečić and M. Krötzsch, "Wikidata: A free collaborative knowledgebase," Commun. ACM, vol. 57, no. 10, pp. 78–85, 2014, doi: 10.1145/2629489.

[36] J. Kim, S. Park, Y. Kwon, Y. Jo, J. Thorne, and E. Choi, "FactKG: Fact verification via reasoning on knowledge graphs," in Proc. ACL, 2023, doi: 10.18653/v1/2023.acl-long.895.

[37] X. Guan et al., "Mitigating large language model hallucinations via autonomous knowledge graph-based retrofitting," in Proc. AAAI, vol. 38, no. 16, pp. 18126–18134, 2024, doi: 10.1609/aaai.v38i16.29770.

[38] K. Lavrinovics, R. Biswas, K. Hose, and J. Bjerva, "MultiHal: Multilingual dataset for knowledge-graph grounded evaluation of LLM hallucinations," arXiv:2505.14101, 2025, doi: 10.48550/arXiv.2505.14101.

[39] G. Agrawal, T. Kumarage, Z. Alghamdi, and H. Liu, "Can knowledge graphs reduce hallucinations in LLMs? A survey," arXiv:2311.07914, 2023, doi: 10.48550/arXiv.2311.07914.

[40] K. Lavrinovics, R. Biswas, J. Bjerva, and K. Hose, "Knowledge graphs, large language models, and hallucinations: An NLP perspective," J. Web Semantics, vol. 85, art. 100844, 2025, doi: 10.1016/j.websem.2024.100844.

[41] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. EMNLP-IJCNLP, 2019, doi: 10.18653/v1/D19-1410.

[42] N. Reimers and I. Gurevych, "Making monolingual sentence embeddings multilingual using knowledge distillation," in Proc. EMNLP, 2020, doi: 10.48550/arXiv.2004.09813.

[43] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," IEEE Trans. Big Data, vol. 7, no. 3, pp. 535–547, 2021, doi: 10.1109/TBDATA.2019.2921572.

[44] P. He, J. Gao, and W. Chen, "DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing," in Proc. ICLR, 2023, doi: 10.48550/arXiv.2111.09543.

[45] Ollama. [Online]. Available: https://ollama.com

[46] OpenAI, "gpt-oss-120b & gpt-oss-20b model card," arXiv:2508.10925, 2025, doi: 10.48550/arXiv.2508.10925.

[47] M. Honnibal, I. Montani, S. Van Landeghem, and A. Boyd, "spaCy: Industrial-strength natural language processing in Python," Zenodo, 2020, doi: 10.5281/zenodo.1212303.

[48] A. Kunchukuttan, "The IndicNLP Library," 2020. [Online]. Available: https://github.com/anoopkunchukuttan/indic_nlp_library

[49] The World Bank, "World Bank Open Data." [Online]. Available: https://data.worldbank.org

[50] World Health Organization, "The Global Health Observatory." [Online]. Available: https://www.who.int/data/gho

[51] Google, "Fact Check Tools API." [Online]. Available: https://developers.google.com/fact-check/tools/api

[52] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Proc. NeurIPS, 2019, arXiv:1912.01703.

[53] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in Proc. EMNLP: System Demonstrations, 2020, doi: 10.18653/v1/2020.emnlp-demos.6.

LIST OF PUBLICATIONS

Published papers and IPR filed in connection with this project:

Sr. No.	Title	Type	Status
1	Satya-Score: An AI Hallucination Detection & Scoring Framework	Research paper (IEEE conference format)	Manuscript prepared; submission pending
2	Satya-Score: An AI Hallucination Detection & Scoring Framework	Copyright (to be filed through the institute)	[Diary / application number, once filed]

RESUME

PERSONAL DETAILS		**Photo**
NAME	Vivek Yeshwantrao Borkar	
DOB		
ADDRESS		
MOBILE NO		
EMAIL_ID		
LINKEDIN ID		

EDUCATION DETAILS					
Name of Board			**Passing Year**	**% of Marks / CGPA**	
10th SSC	Maharashtra State Board				
12th HSC	Maharashtra State Board				
Diploma	Maharashtra State Board of Technical Education				
Bachelor of Technology (B.Tech)		**Passing Year & Month**	**Marks Obtained / Out of**	**% of Marks**	**Pointer**
Ist Year	I - SEM				
	II - SEM				
IInd Year	III - SEM				
	IV - SEM				
IIIrd Year	V - SEM				
	VI - SEM				
IVth Year	VII - SEM				
	VIII - SEM				

PLACEMENT DETAILS	
Campus Placement (If Any)	
(If Any) Name of Company	

FUTURE PLANNING		
Higher Studies / Job Preferences	Higher Studies	
	Job	
	Training	
	Business	

Place: Amravati
Date: / /2026

Signature
Vivek Yeshwantrao Borkar

RESUME

PERSONAL DETAILS		**Photo**
NAME	Piyush Devidas Nande	
DOB		
ADDRESS		
MOBILE NO		
EMAIL_ID		
LINKEDIN ID		

EDUCATION DETAILS					
Name of Board			**Passing Year**	**% of Marks / CGPA**	
10th SSC	Maharashtra State Board				
12th HSC	Maharashtra State Board				
Diploma	Maharashtra State Board of Technical Education				
Bachelor of Technology (B.Tech)		**Passing Year & Month**	**Marks Obtained / Out of**	**% of Marks**	**Pointer**
Ist Year	I - SEM				
	II - SEM				
IInd Year	III - SEM				
	IV - SEM				
IIIrd Year	V - SEM				
	VI - SEM				
IVth Year	VII - SEM				
	VIII - SEM				

PLACEMENT DETAILS	
Campus Placement (If Any)	
(If Any) Name of Company	

FUTURE PLANNING		
Higher Studies / Job Preferences	Higher Studies	
	Job	
	Training	
	Business	

Place: Amravati
Date: / /2026

Signature
Piyush Devidas Nande

RESUME

PERSONAL DETAILS		**Photo**
NAME	Aasawari Sahebrao Jumade	
DOB		
ADDRESS		
MOBILE NO		
EMAIL_ID		
LINKEDIN ID		

EDUCATION DETAILS					
Name of Board			**Passing Year**	**% of Marks / CGPA**	
10th SSC	Maharashtra State Board				
12th HSC	Maharashtra State Board				
Diploma	Maharashtra State Board of Technical Education				
Bachelor of Technology (B.Tech)		**Passing Year & Month**	**Marks Obtained / Out of**	**% of Marks**	**Pointer**
Ist Year	I - SEM				
	II - SEM				
IInd Year	III - SEM				
	IV - SEM				
IIIrd Year	V - SEM				
	VI - SEM				
IVth Year	VII - SEM				
	VIII - SEM				

PLACEMENT DETAILS	
Campus Placement (If Any)	
(If Any) Name of Company	

FUTURE PLANNING		
Higher Studies / Job Preferences	Higher Studies	
	Job	
	Training	
	Business	

Place: Amravati
Date: / /2026

Signature
Aasawari Sahebrao Jumade

RESUME

PERSONAL DETAILS		**Photo**
NAME	Arunima Tridip Wankhade	
DOB		
ADDRESS		
MOBILE NO		
EMAIL_ID		
LINKEDIN ID		

EDUCATION DETAILS					
Name of Board			**Passing Year**	**% of Marks / CGPA**	
10th SSC	Maharashtra State Board				
12th HSC	Maharashtra State Board				
Diploma	Maharashtra State Board of Technical Education				
Bachelor of Technology (B.Tech)		**Passing Year & Month**	**Marks Obtained / Out of**	**% of Marks**	**Pointer**
Ist Year	I - SEM				
	II - SEM				
IInd Year	III - SEM				
	IV - SEM				
IIIrd Year	V - SEM				
	VI - SEM				
IVth Year	VII - SEM				
	VIII - SEM				

PLACEMENT DETAILS	
Campus Placement (If Any)	
(If Any) Name of Company	

FUTURE PLANNING		
Higher Studies / Job Preferences	Higher Studies	
	Job	
	Training	
	Business	

Place: Amravati
Date: / /2026

Signature
Arunima Tridip Wankhade